

Agentic Bayesian Optimization through Surrogate-Augmented Autoresearch

Paul Brunzema^{1,2,*}, Louis Tiao¹, Nhat Le¹, Kevin De Angeli¹, Yao Xuan¹, Djordje Gligorijevic¹

¹Meta, ²RWTH Aachen University, Germany

*Work conducted at Meta during an internship

Bayesian optimization (BO) has become the standard tool for sample-efficient optimization and owes its efficiency to uncertainty-aware search driven by generic statistical priors. Richer domain priors can improve BO in principle, but encoding them through tailored kernels or problem structure is difficult and rarely done in practice. LLMs can help sidestep this difficulty by making informal priors from natural language, code, and documentation directly available to the optimizer. However, existing LLM-based BO methods either insert the LLM into a fixed role—surrogate, acquisition proxy, or configuration interface—or hand it broad control, sacrificing the systematic exploration that makes BO reliable. We introduce **agentic Bayesian optimization**: a paradigm in which an LLM agent is the central decision maker in the BO loop while a Bayesian backend provides the uncertainty-aware optimization substrate. The agent configures the problem, queries the backend, selects and commits evaluations, and can revise the optimization strategy during the run by tightening bounds, switching acquisition functions, proposing targeted evaluations, or even reframing the problem following new instructions or observed evidence. We instantiate this idea in Sara, a surrogate-augmented autoresearch agent, and lenz, a modular BoTorch-based backend that the agent can inspect and modify through a structured interface. Across synthetic and real-world benchmarks, Sara preserves the reliability of state-of-the-art BO without prior knowledge, outperforms LLM-based baselines, and uses natural-language priors to improve beyond standard BO. We further demonstrate the practical value of agentic BO in dynamic settings, where Sara reconfigures the full optimization problem on the fly as requirements change—a capability not previously available in standard BO.

Date: July 31, 2026

Correspondence: brunzema@dsme.rwth-aachen.de, tiao@meta.com



1 Introduction

Bayesian optimization (BO) is the standard tool for sample-efficient, zero-order optimization of expensive black-box functions, with applications ranging from hyperparameter tuning and robotics to experimental design (Snoek et al., 2012; Shahriari et al., 2016; Garnett, 2023) (Figure 1a). Its success comes largely from uncertainty-aware sequential decision making: a probabilistic surrogate, typically a Gaussian process (Rasmussen and Williams, 2006), tracks what is known about the objective, and an acquisition function uses the posterior to select the next point, trading off exploration and exploitation (Jones et al., 1998; Srinivas et al., 2010). There is, however, a fundamental difference between how standard BO and a human approach the same optimization problem. In standard BO the configuration—the surrogate, the acquisition function, the search bounds, and the partition of outcomes into objectives and constraints—is fully specified at the start and held fixed throughout, a setting we refer to as a *fixed* policy, whereas a practitioner performing manual optimization adapts the entire search strategy as new evidence arrives. Approaches that build adaptivity into BO, such as TuRBO (Eriksson et al., 2019) or π BO (Hvarfner et al., 2022), soften this rigidity but do not remove it: their adaptation rules are themselves hand-designed and fixed before the run, so the policy as a whole still does not change in response to what is learned. Recent interactive BO methods further allow structured user priors to be supplied during a campaign (Seng et al., 2025; Fehring et al., 2025). They enable run-time steering, but fix both the form of feedback and the mechanism by which it affects candidate selection.

A second limitation is that classical BO cannot readily absorb the prior knowledge that domain experts

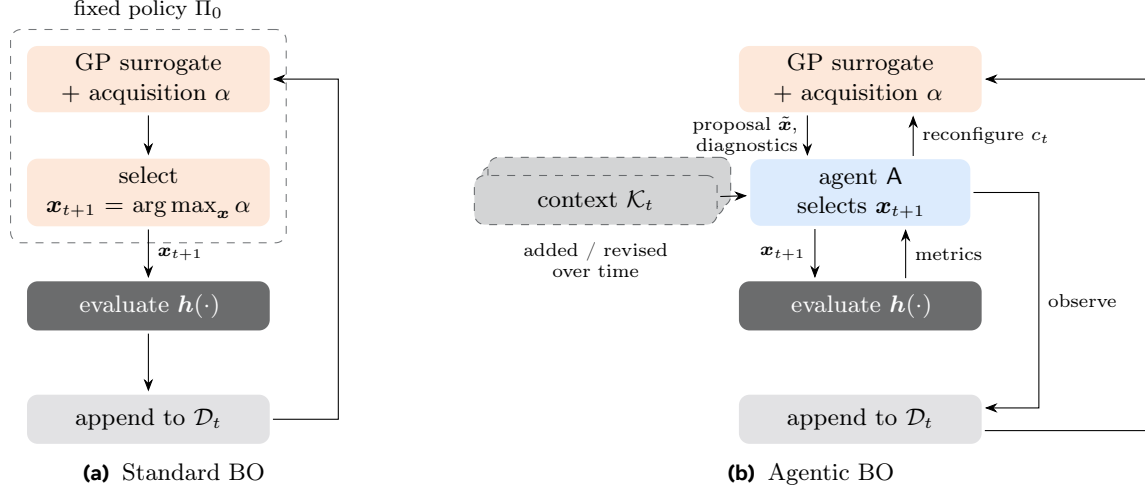


Figure 1 *Standard vs. agentic Bayesian optimization.* In Figure 1a, a fixed policy with configuration c_0 selects the next query point. In Figure 1b, the agent sits at the center of the loop and decides on the next point—adopting, refining, or overriding the surrogate’s proposal $\tilde{x}$ —while also querying surrogate diagnostics and incorporating additional natural-language context or instructions that accrue over the run by reconfiguring the BO backend.

already possess. Problem-specific kernels can accelerate the search (Marco et al., 2017), but they are difficult to derive and can capture only a fraction of the rich, heterogeneous knowledge, often expressed in natural language, code, documentation, and past experimental logs, that practitioners actually hold. Large language models (LLMs) change this picture, as they provide, for the first time, a single model class that can consume all of these sources natively and reason over them jointly, while also deliberating about the optimization as it unfolds. In principle, this makes LLMs ideally suited to drive adaptive, domain-informed black-box optimization.¹

In practice, however, current LLM-based methods realize only part of this promise, and they tend to do so in one of two opposing ways. At one extreme, the LLM replaces the optimization loop entirely. LLM-as-optimizer methods propose points directly from a textual summary of the history (Yang et al., 2024; Zhang et al., 2023a; Liu et al., 2024b,a; Madiraju and Madiraju, 2025), and autoresearch agents assume broad control of scientific workflows (Lu et al., 2024; M. Bran et al., 2023; Boiko et al., 2023; Huang et al., 2023; Zhang et al., 2023b), thereby gaining adaptivity and access to priors but discarding the explicit probabilistic surrogate and associated uncertainty estimates that have traditionally underpinned BO’s sample efficiency in well-defined spaces. At the other extreme, the LLM is embedded as a single component of an otherwise standard loop, *e.g.*, designing the kernel (Suwandi et al., 2026), learning surrogate features or fusing predictions into the surrogate (Ranković et al., 2025; Chen et al., 2026), selecting the acquisition function (Ngo et al., 2026), proposing or scoring candidate points (Cissé et al., 2025; Yang et al., 2025), or translating natural-language feedback and arbitrating between the LLM and the surrogate (Kobalczyk et al., 2026; Chang et al., 2025). All of these preserve a probabilistic surrogate in some form but leave the optimization policy, including all developed heuristics, fixed during optimization. An agent that can read code, form hypotheses, and reason about the search landscape, by contrast, should be able to change *how* it optimizes—tightening bounds, switching acquisition functions, or reframing the problem mid-run—without surrendering the uncertainty quantification that makes the search efficient. None of these approaches fully harnesses the best of both worlds: an LLM that excels at sequential decision making together with a surrogate that supplies uncertainty estimates.

We introduce **agentic Bayesian optimization**, a paradigm in which an LLM agent sits at the center of the optimization loop while delegating probabilistic modeling and acquisition optimization to a Bayesian backend (Figure 1b). This division plays to each component’s strength: the agent contributes semantic priors and judgments that are difficult to encode in standard BO, while the backend provides posterior uncertainty for

¹Throughout the paper, we use *black-box optimization* and *zero-order optimization* interchangeably to denote optimization without access to analytical gradients, not to imply a complete absence of problem knowledge.

systematic, sample-efficient search. Before committing each expensive evaluation, the agent may inspect the surrogate, request, refine, or override proposals, and reconfigure the optimization problem in response to new evidence or changing instructions.

This design follows a broader lesson from the LLM literature, where models become more effective when they can delegate precise computation to specialized tools (Schick et al., 2023; Gao et al., 2023; Chen et al., 2023b). The resulting architecture admits two complementary readings. From the BO perspective, it is *agentic BO*: the agent controls the optimization process while retaining a probabilistic backend. From the autoresearch perspective, it is *surrogate-augmented autoresearch*: the research agent delegates uncertainty-aware numerical search to an explicit surrogate and optimization engine. We instantiate both perspectives in Sara, a Surrogate-augmented autoresearch agent, and lenz (pronounced [lens], as in “lens”), a modular BoTorch-based backend designed for run-time adaptation.

Contributions. In summary, the core contributions of this paper are

- (i) **The paradigm of agentic Bayesian optimization.** We introduce *agentic BO*, in which an agent deliberates over trial data, surrogate diagnostics, and external context to revise the optimization policy during a campaign.
- (ii) **A surrogate-augmented autoresearch system.** We present Sara and lenz, which together instantiate agentic BO. Sara is an LLM agent that drives the optimization process, while lenz is a modular BO backend that exposes BO primitives through a structured command-line interface.
- (iii) **An empirical study of agentic BO.** Across synthetic and real-world benchmarks, Sara matches state-of-the-art BO performance without prior knowledge, outperforms LLM-based BO methods such as LLAMBO, and uses natural-language priors to improve beyond Ax. We further demonstrate its practical value in dynamic settings, where Sara reconfigures the optimization problem as requirements change.

2 Background and Problem Setting

We first introduce the class of problems we consider and then frame standard Bayesian optimization as a fixed policy defined before the campaign begins.

Problem. We aim to optimize an expensive-to-evaluate black-box function over a domain $\mathcal{X} \subseteq \mathbb{R}^d$. Each evaluation yields a vector of noisy outcomes $\mathbf{y}_i = \mathbf{h}(\mathbf{x}_i) + \boldsymbol{\varepsilon}_i$, where $\mathbf{h} : \mathcal{X} \rightarrow \mathbb{R}^p$, and after t evaluations the collected data is $\mathcal{D}_t = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^t$. A problem specification assigns components of $\mathbf{h}$ to objective functions f_o , indexed by $o \in \mathcal{O}$, and constraint functions $c \in \mathcal{C}$, with each constraint expressed as $c(\mathbf{x}) \leq 0$. This gives the constrained multi-objective problem

$$\max_{\mathbf{x} \in \mathcal{X}} (f_o(\mathbf{x}))_{o \in \mathcal{O}} \quad \text{s.t.} \quad c(\mathbf{x}) \leq 0 \quad \forall c \in \mathcal{C}, \quad (2.1)$$

with the maximum understood in the Pareto sense. The familiar cases are recovered by choice of $\mathcal{O}$ and $\mathcal{C}$, *i.e.* single-objective BO as $|\mathcal{O}| = 1$, $\mathcal{C} = \emptyset$, constrained BO as $\mathcal{C} \neq \emptyset$, and multi-objective BO as $|\mathcal{O}| > 1$. Which outcomes serve as objectives and which as constraints is a modeling choice, not a property of the experiment itself.

The standard BO loop. Bayesian optimization fits a probabilistic surrogate to $\mathcal{D}_t$ and uses it to select the next query. Placing a GP on each outcome gives a posterior with mean μ_t and variance σ_t^2 (Rasmussen and Williams, 2006), and an acquisition function $\alpha(\mathbf{x}; \mathcal{D}_t)$ scores candidate points by their expected utility under this posterior. It is useful to separate two steps that the standard loop conflates: the acquisition function *proposes* by ranking $\mathcal{X}$, and a selection rule *commits* one candidate as the next evaluation. Standard BO uses the arg max rule,

$$\mathbf{x}_{t+1} = \arg \max_{\mathbf{x} \in \mathcal{B}} \alpha(\mathbf{x}; \mathcal{D}_t), \quad (2.2)$$

over an active search region $\mathcal{B} \subseteq \mathcal{X}$. We keep these two steps distinct in what follows, since an agent can retain the surrogate’s proposal while overriding the selection.

	Calibrated surrogate	Full run-time policy control	Natural-language priors & tool use
Classical BO & AutoML (Ax, TPE, OptFormer, ...)	✓	✗	✗
Interactive BO (IBO-HPC, DynaBO, ...)	✓	✗	✗
Hybrid LLM-BO (LILO, BORA, Reasoning BO, ...)	✓	✗	✓
LLM-as-optimizer (LLAMBO, OPRO, AgentHPO, ...)	✗	✓	✓
Open-ended autoresearch agents (HyperAgents, ...)	✗	✓	✓
Agentic BO / Surrogate-augmented autoresearch (Sara + lenz)	✓	✓	✓

Table 1 *Agentic BO in the optimization landscape.* Classical BO and AutoML ground every decision in a probabilistic surrogate but run a *fixed* policy. LLM-as-optimizer methods and open-ended autoresearch agents exercise run-time control but discard posterior uncertainty. Sara retains access to a BO backend lenz *and* reasons about the search.

Policy and configuration. More broadly, the full set of choices active at a given campaign step—the model class $\mathcal{M}_t$, acquisition function α_t , active region $\mathcal{B}_t$, and problem partition $(\mathcal{O}_t, \mathcal{C}_t)$ —form the **configuration**

$$c_t = (\mathcal{M}_t, \alpha_t, \mathcal{B}_t, \mathcal{O}_t, \mathcal{C}_t). \quad (2.3)$$

A standard BO loop similar to the one described above then amounts to a fixed policy Π_0 that maps the accumulated data to the active configuration and next query,

$$(c_t, \mathbf{x}_{t+1}) = \Pi_0(\mathcal{D}_t), \quad \Pi_0 \text{ fixed before the campaign.} \quad (2.4)$$

Here, *fixed* refers to the mapping Π_0 , but not its outputs, *i.e.*, the configuration may change during the campaign, but only according to rules prescribed in advance. Modern BO frameworks (Olson et al., 2025; Akiba et al., 2019) are concrete instances of such a policy. For example, Ax switches from Sobol initialization to model-based search, but both the switching rule and the search strategy are determined before the first evaluation; similarly, most specialized BO algorithms are fully specified by heuristics at startup.

3 Related Work

Our work sits at the intersection of BO, automated machine learning (AutoML), LLM-driven optimization, and autonomous research agents. We organize prior work by whether and where the LLM enters the optimization loop: not at all (classical BO and AutoML), as a single *fixed* component of it (hybrid LLM-BO), *in place of* it (LLMs as the optimizer), or *around* a broader workflow (open-ended agents). Table 1 organizes these by whether decisions can be supported by a probabilistic surrogate, how much run-time control the method exerts over its own policy, and whether it can consume natural-language priors and call tools.

3.1 Bayesian optimization and AutoML

Classical BO couples a probabilistic surrogate, typically a GP (Rasmussen and Williams, 2006), with an acquisition function that balances exploration and exploitation (Jones et al., 1998; Srinivas et al., 2010; Shahriari et al., 2016; Frazier, 2018; Garnett, 2023). Mature frameworks such as Ax (Olson et al., 2025) or Optuna (Akiba et al., 2019) package these components into reusable routines, but by design the optimization policy is declared before the campaign and held fixed throughout. Notable extensions equip BO with adaptive mechanisms: TuRBO (Eriksson et al., 2019; Eriksson and Poloczek, 2021) restricts the search to a local region that expands or contracts with progress, while user priors can bias early search through the acquisition function (Hvarfner et al., 2022). Both are *hand-designed* adaptation rules for a specific situation. Interactive BO extends this idea by allowing users to steer the search during a campaign. IBO-HPC conditions candidate generation on user-specified beliefs through probabilistic circuits (Seng et al., 2025), while DynaBO incorporates successive priors and safeguards against misleading ones (Fehring et al., 2025). Both retain a probabilistic surrogate but restrict interaction to structured priors interpreted by a fixed selection rule. Agentic BO broadens this interaction by allowing an agent to reason over unstructured context and the evolving campaign, and to adapt the optimization process accordingly. Beyond BO, the broader AutoML

literature—from random search (Bergstra and Bengio, 2012) and tree-structured estimators (Bergstra et al., 2011) to multi-fidelity methods (Li et al., 2018) and meta-learned optimizers (Chen et al., 2022)—similarly commits to a fixed search strategy in advance (Tornede et al., 2024). Sara keeps the explicit probabilistic model but inverts the control relationship: *which* strategy to apply, and *when*, becomes a decision made by reasoning over the run, informed by the priors extracted from the instructions and the observations gathered during it.

3.2 Hybrid LLM-BO methods

A growing line of work keeps the Bayesian machinery but hands a *single, fixed* component of the loop to an LLM; methods differ in which component that is. Some target the *kernel*, using an LLM to design or adapt the GP covariance from a contextual description of the problem (Suwandi et al., 2026). Others reshape the *surrogate* itself, training LLM embeddings through the GP marginal likelihood so that uncertainty stays calibrated (Ranković et al., 2025), or fusing computationally cheap LLM predictions with real evaluations in a multi-fidelity surrogate (Chen et al., 2026). A third line acts at the *acquisition* step, letting the LLM select the acquisition function from a portfolio at each iteration (Ngo et al., 2026). A fourth group acts on candidate selection, letting the LLM propose or filter candidates (Cissé et al., 2025; Yang et al., 2025) or arbitrate between LLM and GP proposals (Chang et al., 2025). Finally, Kobalczyk et al. (2026) intervene on outcome interpretation, translating natural-language feedback into preferences for a preference GP. Agarwal et al. (2025) go further, running BO in a latent embedding space where acquisition proposals bias in-context examples for LLM generation, effectively turning the LLM into a conditioned decoder from surrogate proposals to candidate text. Together these results show that an LLM and a probabilistic model are complementary, and several report strong gains in the early, low-data regime. In every case, however, the point of intervention—and the policy around it—is fixed before the first evaluation and the LLM essentially fills one slot in a standard pipeline. Even the methods that invoke the LLM adaptively do so through a hand-designed trigger, *e.g.* a heuristic on GP uncertainty and performance plateaus (Cissé et al., 2025), or a fixed probabilistic switching schedule (Mahammadli and Ertekin, 2024; Chang et al., 2025), rather than leaving the model itself to decide when to intervene. Agentic BO instead places the agent *at the center* of the loop, where it can reconfigure any component—bounds, objectives, constraints, acquisition behavior, even the objective/constraint partition—at run time, while delegating calibrated numerical search to lenz. Importantly, the methods above are complementary to, not superseded by, agentic BO, since an LLM-designed kernel (Suwandi et al., 2026), a latent-space generation strategy (Agarwal et al., 2025), or a preference surrogate (Kobalczyk et al., 2026) can each be wrapped as a backend capability that the agent invokes when appropriate, and the system prompt can incorporate the acquisition selection strategy of Ngo et al. (2026). The contribution of the agentic layer is orthogonal; it decides *when* and *how* to combine such capabilities in response to accumulating evidence, rather than fixing the choice before the first evaluation.

3.3 LLMs as the optimizer

At the other extreme, the optimization itself moves inside the LLM, an approach advocated as a general direction for black-box problems (Song et al., 2024). OPRO (Yang et al., 2024) and related methods prompt the model with a trajectory summary and ask it to emit the next configuration directly (Yang et al., 2024; Zhang et al., 2023a); program-search methods evolve solutions from LLM-proposed edits (Romera-Paredes et al., 2024); and LLAMBO (Liu et al., 2024b) frames this as BO by prompting for a promising point. None maintains an explicit probabilistic surrogate, so their search is not guided by posterior uncertainty over the objective. While LLM processes (Requeima et al., 2024) show that LLMs can form predictive distributions, a large body of work finds that their uncertainties can be poorly calibrated (Kadavath et al., 2022; Tian et al., 2023; Xiong et al., 2024), including for molecular BO specifically (Kristiadi et al., 2024). Recent evidence reinforces the value of a dedicated optimization substrate. Ferreira et al. (2026) find that classical HPO methods (CMA-ES, TPE) outperform pure-LLM optimizers on a language-model tuning task, and similar findings question whether LLMs are ready for scientific BO on their own (Gupta et al., 2025). On this basis, Ferreira et al. (2026) build Centaur, a hybrid that shares CMA-ES internal state with an LLM through its prompt. Centaur is the closest baseline in spirit to agentic BO; however, augmenting a prompt underutilizes the sequential decision-making of LLM agents. We instead give Sara full control over a BO backend and show substantially better performance than Centaur in our experiments.

3.4 LLM agents for optimization and ML research

A rapidly growing line of work wraps LLMs in agent loops to drive ML workflows. AgentHPO (Liu et al., 2024a) reads a task description, proposes configurations, and refines them from past trials; OptiMindTune (Madiraju and Madiraju, 2025) splits the job across cooperating agents; others orchestrate whole pipelines (Zhang et al., 2023b), generate features (Hollmann et al., 2023), or search architectures by evolving code (Chen et al., 2023a), and benchmarks now probe how well LLMs act as ML research agents (Huang et al., 2023). Like the in-context methods above, these drive search by prompting over text summaries of past trials with no calibrated posterior. Agentic BO differs in that a calibrated surrogate can inform every decision the agent makes. We believe that agentic BO could be readily incorporated in the above approaches. More broadly, general reasoning-and-acting frameworks (Yao et al., 2023), tool-use paradigms (Schick et al., 2023), multi-agent systems (Hong et al., 2024), open-ended embodied agents (Wang et al., 2023), and end-to-end scientific discovery pipelines (Lu et al., 2024; Boiko et al., 2023; M. Bran et al., 2023) emphasize broad autonomy. A recurring lesson is that LLMs are unreliable at direct computation but effective when they *delegate* to precise tools (Gao et al., 2023; Chen et al., 2023b; Schick et al., 2023). Sara applies this principle to zero-order optimization, informing her search with a BO backend while remaining deliberately narrower than open-ended agents.

4 Agentic Bayesian Optimization

To our knowledge, no existing approach combines three capabilities: (i) retaining a probabilistic surrogate, (ii) adapting the optimization strategy in response to accumulating evidence, and (iii) interpreting and incorporating unstructured domain knowledge or changing task instructions during the campaign.

We introduce agentic BO as an implementation-agnostic framework that combines these capabilities by placing an agent in control of a BO backend. The framework specifies the information available to the agent, the computational and evaluation actions it may take, and how these actions update or reconfigure the optimization campaign. Formally, agentic BO extends the fixed-policy loop in (2.4) by replacing its prescribed acquisition step with an agent-controlled *deliberation process* over a Bayesian backend. Before committing an expensive evaluation, the agent may inspect the surrogate, request and refine proposals, or persistently reconfigure the optimization strategy. We model this interaction as a *metalevel decision process*, inspired by metalevel MDPs (Hay et al., 2012; Callaway et al., 2022). The formulation captures two coupled decisions: which computational actions to take and which point to evaluate. Throughout, *deliberation* refers to the observable sequence of computational actions and backend responses. In Section E, we show how our formulation can be completed into a full meta-MDP that could form the basis for training or fine-tuning a specialized agentic BO policy. The goal of this paper, however, is to show that a general-purpose LLM can already implement an effective agentic BO policy without specialized post-training.

State. Let t index expensive evaluations and j index computational actions taken between evaluations. During deliberation, the agent observes the state

$$s_t^{(j)} = (\mathcal{D}_t, c_t^{(j)}, \mathcal{K}_t, \mathcal{H}_t^{(j)}), \quad (4.1)$$

where $\mathcal{D}_t$ is the observed trial data, $c_t^{(j)}$ is the current optimization configuration (*cf.* (2.3)), $\mathcal{K}_t$ is the exogenous context, and $\mathcal{H}_t^{(j)}$ is the deliberation history available to the agent. The context may include the problem description, domain knowledge, and requirements supplied by an instructor, such as a human or another agent. It is append-only: new information or revised requirements are added during the campaign, so $\mathcal{K}_{t+1} \supseteq \mathcal{K}_t$, with later instructions able to semantically supersede earlier ones. The history records the retained conversation, reasoning trace, tool calls and responses, proposals, and past interventions. Although separated conceptually, messages comprising $\mathcal{K}_t$ also appear in the conversational history $\mathcal{H}_t^{(j)}$ in implementations such as Sara (*cf.* Section 5). Other instantiations may instead expose this context through external memory that the agent accesses during deliberation.

Action space. At each deliberation step, the agent chooses among three classes of actions,

$$\mathcal{A} = \mathcal{A}_{\text{comp}} \cup \mathcal{A}_{\text{eval}} \cup \{\text{stop}\}, \quad (4.2)$$

which differ in their effect on the state and the black-box evaluation budget:

- (i) **Evaluate** ($\mathcal{A}_{\text{eval}}$). The agent commits a point $\mathbf{x}_{t+1} \in \mathcal{X}$ to the expensive oracle. This is the only action that advances the campaign counter t and consumes black-box evaluation budget.
- (ii) **Computational** ($\mathcal{A}_{\text{comp}}$). Backend interactions in this class do *not* trigger an expensive evaluation and can inform the agent’s next decision. These actions are free with respect to the black-box evaluation budget and do not advance t , although they incur computational overhead (tokens, context length, and backend compute).² We further partition them into three main classes:
 - **probe**: read-only queries to the backend—predictions, acquisition scores, diagnostics, the current Pareto front, or trial history—whose responses are added to $\mathcal{H}_t^{(j)}$;
 - **reconfigure**: mutations of $c_t^{(j)}$, *e.g.*, tightening the search region $\mathcal{B}$, swapping the acquisition function α , or revising objectives $\mathcal{O}$ and constraints $\mathcal{C}$;
 - **propose**: requests for one or more candidate points $\tilde{\mathbf{x}}$ from the surrogate backend via acquisition-function optimization over the current active region.
- (iii) **Stop** (stop). Terminate the campaign and return the incumbent feasible solution in the single-objective case or the current feasible Pareto set in the multi-objective case.

A computational action returns an observation $o_t^{(j)}$ from the backend, and its call and response are appended to the deliberation history. A reconfiguration additionally updates $c_t^{(j)} \mapsto c_t^{(j+1)}$. None of these operations changes $\mathcal{D}_t$ or advances the evaluation counter t . Deliberation is therefore broader than reconfiguration: probe and propose actions can change the eventual evaluation decision without mutating $c_t^{(j)}$, whereas a reconfigure action persistently changes how subsequent backend calls are interpreted or optimized.

Metalevel policy and deliberation. The agent implements a metalevel policy $\mathbf{A} : \mathcal{S} \rightarrow \mathcal{A}$, which is applied repeatedly to the evolving deliberation state $s_t^{(j)}$ until an evaluation or stop action is selected. Each computational action and its response update the state on which the next action is conditioned, allowing the tool-use strategy to adapt within a campaign step. A campaign step consists (*cf.* Figure 2) of $k_t \geq 0$ computational actions followed by an evaluation or termination:

$$a_t^{(0)}, \dots, a_t^{(k_t-1)} \in \mathcal{A}_{\text{comp}}, \quad a_t^{(k_t)} \in \mathcal{A}_{\text{eval}} \cup \{\text{stop}\}. \quad (4.3)$$

If $a_t^{(k_t)} = \mathbf{x}_{t+1} \in \mathcal{A}_{\text{eval}}$, the oracle returns $\mathbf{y}_{t+1} = \mathbf{h}(\mathbf{x}_{t+1}) + \varepsilon_{t+1}$ and the campaign advances as

$$\mathcal{D}_{t+1} = \mathcal{D}_t \cup \{(\mathbf{x}_{t+1}, \mathbf{y}_{t+1})\}, \quad c_{t+1}^{(0)} = c_t^{(k_t)}. \quad (4.4)$$

New exogenous information may then be appended to form $\mathcal{K}_{t+1}$, and the deliberation history carries forward to the next campaign step. If instead $a_t^{(k_t)} = \text{stop}$, the campaign terminates.

We additionally distinguish an initial **setup** phase ($t = 0$) in which the agent formalizes the problem from $\mathcal{K}_0$, *i.e.*, it defines $\mathcal{X}$, partitions outcomes into $\mathcal{O}$ and $\mathcal{C}$, and initializes $c_0^{(0)}$. We write the complete deliberation induced by $\mathbf{A}$ as the campaign-level mapping

$$(c_{t+1}^{(0)}, \mathbf{x}_{t+1}) = \text{Rollout}(\mathbf{A}, s_t^{(0)}) \quad (4.5)$$

with the understanding that this subsumes the deliberation sequence (4.3) and may instead return stop. Ultimately, it is the *agent* that commits $\mathbf{x}_{t+1}$. To form this decision, it may obtain one or more candidate proposals $\tilde{\mathbf{x}}$ from the surrogate and then adopt, refine, or override them.

²We report token usage in Section C.2 but do not optimize for computational cost in this work. A full meta-MDP may account for these computational costs explicitly, as discussed in Section E.

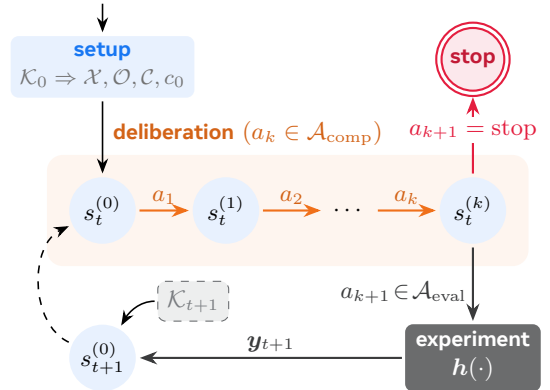


Figure 2 One campaign step in agentic BO. Computational actions inform or reconfigure the search before the agent evaluates or stops.

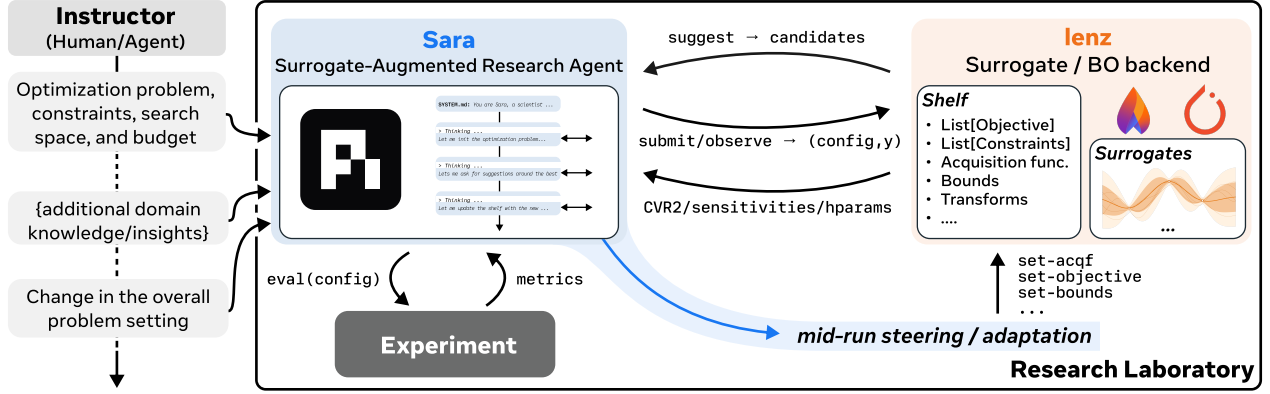


Figure 3 Our proposed instantiation of agentic Bayesian optimization. Sara sits at the center of the optimization loop: she reads trial data and surrogate diagnostics, incorporates natural-language context, queries the surrogate, adapts the optimization policy mid-run, and commits the next evaluation.

This formulation includes several familiar regimes as special cases:

- without backend interactions, the agent proposes evaluations directly, recovering LLM-as-optimizer methods (Liu et al., 2024b,a);
- with a fixed configuration and one surrogate proposal that is always accepted per campaign step, it recovers the standard BO loop (2.4);
- with probes, reconfigurations, multiple proposals, or agent overrides, it yields full agentic BO.

The full interaction provides two levels of control beyond the standard loop (also see Figure 1). At the *metalevel*, the agent chooses a response-conditioned sequence of computational actions rather than invoking a prescribed acquisition rule once; this includes, but does not require, making $c_t^{(j)}$ time-varying through reconfiguration. At the *evaluation level*, the final decision on $\mathbf{x}_{t+1}$ rests with the agent rather than a fixed arg max rule. Consequently, agentic adaptation can be visible in changing probe and proposal patterns even when the persistent optimization configuration of the backend remains fixed.

The formulation is deliberately agnostic to implementation choices. It prescribes neither the agent’s instantiation, nor the surrogate class, nor how $\mathcal{K}_t$ is produced or delivered. Any system in which the agent observes $s_t^{(j)}$, chooses computational actions through $\mathcal{A}_{\text{comp}}$, and ultimately commits an evaluation query—while retaining or changing the configuration—is an instance of agentic BO. The next section describes one such instantiation built around a reasoning LLM and a BO backend.

5 Sara and lenz as the surrogate-augmented research team

The paradigm of Section 4 requires two ingredients: a flexible backend that the agent can query, steer, and reconfigure without losing data, and an agent that knows when to use each capability. We instantiate the backend as lenz (Section 5.1), a modular system based on GPyTorch (Gardner et al., 2018) and BoTorch (Balandat et al., 2020).³ Its interface supports setup and the three computational action classes of agentic BO. We instantiate the agent as Sara (Section 5.2), a reasoning LLM that drives the optimization and uses lenz’s surrogates and BO engine, as shown in Figure 3.

5.1 lenz: a highly modular backend for agentic Bayesian optimization

For (4.5) to be practical, the backend must support the full deliberation phase, *i.e.* probing, proposing, and reconfiguring, without invalidating prior evaluations. Changing c_t should reuse every prior evaluation rather than restart the campaign, and the interface must be intuitive enough for an agent to operate from a single

³GPyTorch (<https://gpytorch.ai/>) and BoTorch (<https://botorch.org/>) are available under the MIT license.

reference sheet. lenz achieves this by making the raw trial log the single source of truth and exposing a command-line interface (CLI) to the surrogates and to established BO routines. Adding a new constraint metric, tightening bounds, or switching the objective therefore never invalidates collected data. Figure 4 sketches the resulting architecture. A *Frame* holds the trial log $\mathcal{D}_t$, the event log $\mathcal{E}_t$, and the mutable *shelf* containing the configuration c_t , while the GP surrogates are derived objects rebuilt from $\mathcal{D}_t$ on each query. Because the state file remains the sole source of truth, lenz can keep its engine resident in a per-study daemon with no risk of cache incoherence. We next describe this CLI in detail.

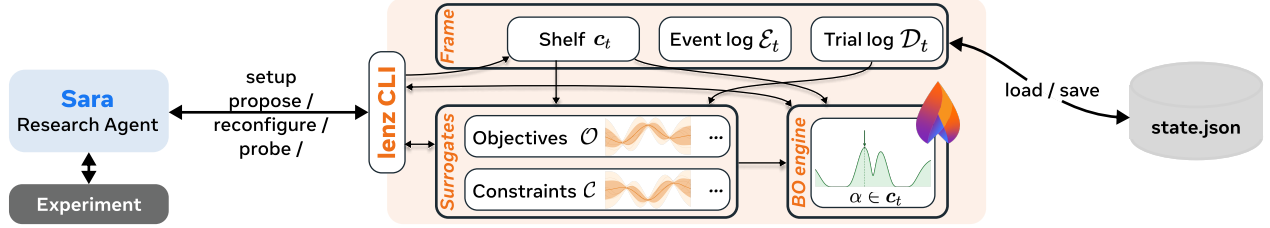


Figure 4 Schematic overview of lenz. All internal functionality is exposed through a single CLI.

Interface design. lenz exposes its BO primitives through a small, role-structured CLI organized around the action categories introduced in Section 4. Below we list the main commands accessible to an agent, grouped by the role they play in the agentic BO loop.

- (i) **setup (create)**. Before the loop begins, the agent declares the search space, objectives, constraints, and initial acquisition function based on the initial instructions $\mathcal{K}_0$:

```
lenz create --space '{"lr":{"kind":"range","lower":1e-4,"upper":1e-1,"log_scale":true}, ...}'
--objectives '{"loss":"minimize"}'
--constraints '{"metric":"flops","upper":500}'
--acqf noisy_logei
```

This materializes the initial configuration c_0 .

- (ii) **propose and evaluate (suggest, submit, observe)**. The agent can request candidates from the surrogate via **suggest**, which may be called multiple times without consuming evaluation budget. When the agent commits a configuration for external evaluation, **submit** marks it as *pending*; subsequent **suggest** calls fantasize an outcome for that configuration to ensure batch diversity. After the experiment, **observe** records the returned metrics in the trial log.

```
lenz suggest --q N # suggest a batch of points
lenz suggest --around '{"lr":0.1, "dropout":{"fix":0.0}, ...}' # around incumbent
lenz suggest --bounds '{"lr":[0.001,0.1],"x2":[10,20]}' # within provided bounds
lenz submit --config '{...}' # commit to a run for fantasization
lenz observe --config '{...}' --metrics '{...}' # record metrics
```

Because **suggest** only queries the surrogates and BO engine, the agent can compare hypothetical proposals under different bounds or acquisition functions before committing.

- (iii) **reconfigure (set-bounds, set-acqf, set-objectives, set-constraints)**. The agent can edit the full specification of the optimization problem at every time step:

```
lenz set-bounds --bounds '{"lr":[1e-4,1e-3]}' # refine search bounds
lenz set-acqf --acqf ucb --beta 2.0 # change acquisition function (and its hyperparameter)
lenz set-objectives --objectives '{"loss":"min","flops":"min"}'
lenz set-constraints --constraints '{"metric":"flops","upper":700}'
```

Because all outcomes are stored independently, they are not tied to a specific role in the optimization specification. We will demonstrate this flexibility in an example moving from constrained single-objective to multi-objective Pareto-front optimization without restarting in [Section 6.4](#).

- (iv) **probe** (diagnostics, predict, score, incumbent, pareto, trials, status). These read-only commands expose surrogate diagnostics, predictions, acquisition scores, incumbents, the current Pareto front, and trial history to inform the agent’s decisions or further deliberation.

```
lenz diagnostics # outputs CVR2, all the GP hyperparameters, as well as sensitivities
lenz predict --configs '[]' # outputs posterior estimates + probability of feasibility
lenz score --configs '[]' --acqf logei # scores the configs according to the acq. function
lenz incumbent # returns the current incumbent
lenz pareto # returns the current pareto front
lenz trials # returns the full list of conducted experiments
lenz status # reports the current study and pending trials
```

The **diagnostics** command helps the agent assess *when to trust* the surrogate by returning cross-validated R^2 and sensitivity estimates for every dimension, signals that BO practitioners also use to diagnose optimization loops.

To make lenz output easy to parse, every command returns JSON. [Section H](#) documents the concrete output for each command.

Surrogate and acquisition details. Under the hood, lenz uses GP surrogates with input normalization and output standardization, following standard BO practice ([Balandat et al., 2020](#)). The backend can be extended with other model classes, such as neural network-based models ([Li et al., 2024](#); [Brunzema et al., 2025](#)), allowing Sara to perform explicit model selection. For constrained optimization, lenz initially optimizes the probability of feasibility and switches to constrained logEI after finding a feasible solution ([Gardner et al., 2014](#); [Gelbart et al., 2014](#); [Ament et al., 2023](#)). For multi-objective optimization, we include acquisition functions based on expected hypervolume improvement for both the noisy and deterministic cases ([Daulton et al., 2020, 2021](#)). For batch calls to **suggest**, lenz uses fantasization to ensure batch diversity. As stated above, for all acquisition functions the optimization bounds can be adjusted by the agent at any point to focus on promising regions.

5.2 Sara: the surrogate-augmented autoresearch agent

Sara is the reasoning agent that drives the optimization and interacts with lenz. Her behavior is governed by the underlying LLM, a system prompt defining her as a surrogate-augmented autoresearch and BO agent, and the reference documentation for lenz. We provide the complete **SYSTEM.md** and **LENZ_REF.md** in [Sections F](#) and [G](#), respectively. To have full control over the system prompt, we instantiate Sara in the harness pi⁴ as

```
pi --tools read,bash --system-prompt "$(cat SYSTEM.md)" --append-system-prompt "$(cat LENZ_REF.md)"
```

Beyond the choice of LLM, we found that the design of the system prompt does affect optimization quality. We provide the full prompt in [Section F](#) and summarize its most important directives here in [Table 2](#).

6 Experiments

We evaluate Sara and lenz on standard synthetic BO functions, the LCBench AutoML benchmark, and a family of synthetic chemical-process optimization problems. These experiments address the following research questions.

- (Q1) Without any prior knowledge, can agentic BO recover standard BO performance and outperform pure LLM-based approaches?

⁴pi (<https://pi.dev/>) is available under the MIT license.

Table 2 Core directives in Sara’s system prompt and their motivation (full system prompt in [Section F](#)).

Directive	Motivation
Agent-led, surrogate-assisted posture. Sara owns the optimization decisions, including framing the problem, deriving priors, and choosing candidates, while treating lenz as an instrument for calibrated search, not an autopilot.	We observed that without explicit ownership, the agent can become lazy and defer all decisions to the surrogate, reducing the loop to vanilla BO.
Sequential reasoning per trial. ¹ Each suggest → evaluate → observe cycle is framed as a new reasoning step; the agent must interpret the result before running the next candidate.	Similar to above, without per-trial reflection the agent can revert to running a full bash loop without any intermediate reasoning.
Adaptive opening strategy. The first evaluations are chosen based on signal strength: a concrete configuration when strong priors exist, a bounded region when only a region is trusted, and space-filling points when no prior is available.	A fixed opening (<i>e.g.</i> always Sobol) wastes budget when strong beliefs exist. We observed that Sara tended to warm up the surrogate with Sobol samples before testing her own hypothesis.
Explicit anti-patterns. We enumerate specific failure modes: deferring to an untrustworthy posterior, recording predictions instead of observations, and discarding context signals.	Each failure mode was observed in early experiments, though only rarely; explicit prohibition was effective in preventing these behaviors.
Reasoning visibility. Before every submission the agent must state what it believes and what the next evaluation should learn.	Explicit verbalization anchors the decision and enables post-hoc auditing.

¹ This directive mitigates but does not eliminate the behavior. Models trained as coding agents retain a turn-economy prior and, given `bash`, still issue batched and looped `suggest` calls—trading surrogate-update frequency for fewer turns. Fully enforcing one reasoning step per evaluation would require harness-level constraints in lenz rather than prompting alone.

(Q2) Can Sara turn a natural-language problem description into a useful domain-informed prior, reflected in a better warm start or faster convergence?

(Q3) How does Sara adapt its use of the Bayesian backend during a campaign, and can it reformulate the problem when requirements change?

(Q4) How important are model family, capability, and reasoning level for the final performance?

On the experimental setup. Standard test functions (Ackley, Hartmann, ...) and their optima are well-represented in LLM pretraining corpora. To prevent LLM-based methods from directly exploiting memorized solutions, every synthetic benchmark is presented through an opaque programmatic interface: parameters are renamed, the domain is canonicalized to the unit hypercube, and optima are shifted away from their textbook locations. We observed that without these measures, LLAMBO and Sara were able to one-shot problems such as Ackley whose optimum lies at the center of the domain. Because Sara executes experiments autonomously and can inspect her working directory, we place each run in a sandbox that contains only the task description, a random token, and a symbolic link to the evaluation oracle; the sandbox is named by the token rather than the benchmark. Neither the path nor the directory contents reveal which benchmark she is solving. In all experiments, we report *simple regret* (best-so-far gap to the known optimum, or raw best value for yield objectives) as a function of evaluation count, aggregated over 10 independent seeds. All plots show the median with a 25–75% interquartile band. [Section A](#) provides full setup details.

Baselines. We compare against four baselines (full details in [Section B](#)):

- **Sobol** quasi-random sampling, as a space-filling lower bound.
- **Ax** (Olson et al., 2025) with its default strategy, representing a well-tuned classical BO policy with best practices implemented for all problem classes.
- **LLAMBO** (Liu et al., 2024b), which renders the trial history as a table and queries an LLM for the next candidate directly, without maintaining a calibrated surrogate model.
- **Centaur** (Ferreira et al., 2026), a recent hybrid approach similar to our surrogate-augmented research agent that incorporates CMA-ES (Hansen, 2016) internal state into the context of an LLM autoresearcher.

When a natural-language problem description is available, it is provided to Sara, LLAMBO, and Centaur,

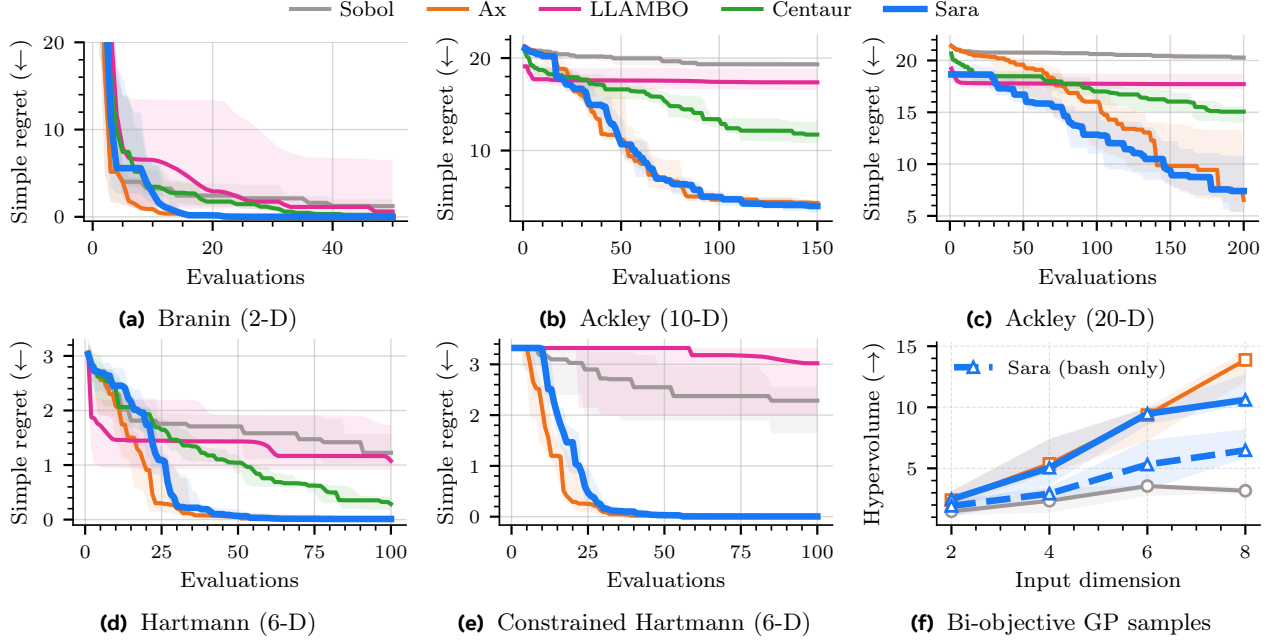


Figure 5 Synthetic benchmarks. Simple regret vs. evaluations for the single-objective and constrained problems; final hypervolume vs. input dimension for the multi-objective sweep. Median over 10 seeds; shaded regions indicate the 25–75% interquartile range.

ensuring the comparison isolates algorithmic differences rather than information asymmetry. Unless otherwise specified, all LLM-based baselines use Claude Opus 4.8 with *high* reasoning mode.

6.1 Synthetic benchmarks

We first evaluate on synthetic problems where the language model receives no domain description. This setting is adversarial for our approach in the sense that the agent has no semantic information to exploit. The benchmark suite spans low-dimensional (Branin, 2-D), moderate-dimensional (Hartmann, 6-D), and high-dimensional (Ackley, 10-D and 20-D) unconstrained problems, as well as a constrained problem (constrained Hartmann, 6-D) and a multi-objective dimensional sweep. The latter minimizes two independent GP sample paths and measures hypervolume relative to $(0, 0)$; construction details are given in [Section A.1](#).

Results are shown in [Figure 5](#). Sara performs on par with Ax across the single-objective benchmark suite: she matches Ax on the low- and moderate-dimensional problems and slightly outperforms it on Ackley (20-D), where her initial space-filling strategy and adaptive search-space refinement yield faster convergence. The multi-objective sweep shows an increasing advantage of the full system over a **bash**-only variant of Sara as dimensionality grows (more discussion on this baseline in [Section 6.5](#) and [Section D.1](#)). The LLM-in-the-loop baselines substantially underperform on these problems, at times performing worse than random sampling. Without recognizable structure in the trial history, their LLM-generated proposals are effectively uninformed, and these methods fail to explore the search space systematically. In contrast, Sara uses lenz’s surrogate-backed search on these tasks, incurring no apparent penalty from the additional agentic layer. This confirms that the framework gracefully reduces to standard BO performance when the agent has no useful domain knowledge to contribute.

6.2 Hyperparameter optimization

We next consider a setting where domain knowledge is available and optimize neural network hyperparameters on LCBench ([Zimmer et al., 2021](#)). Sara and the other LLM-based baselines receive a natural-language description of the overall training setup (see [Section A](#)).

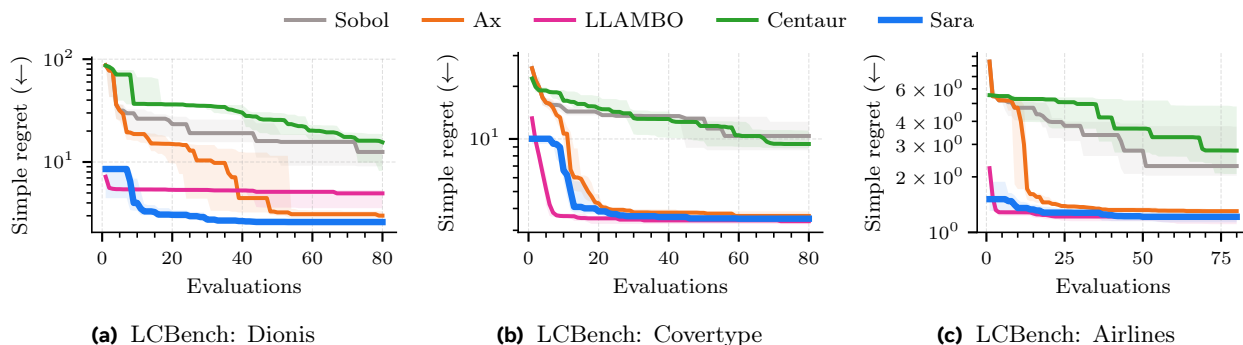


Figure 6 *Hyperparameter optimization on LCBench.* Simple regret (log scale) vs. evaluations. Sara and LLAMBO achieve strong initial performance. The final performance for Sara, LLAMBO, and Ax is very similar.

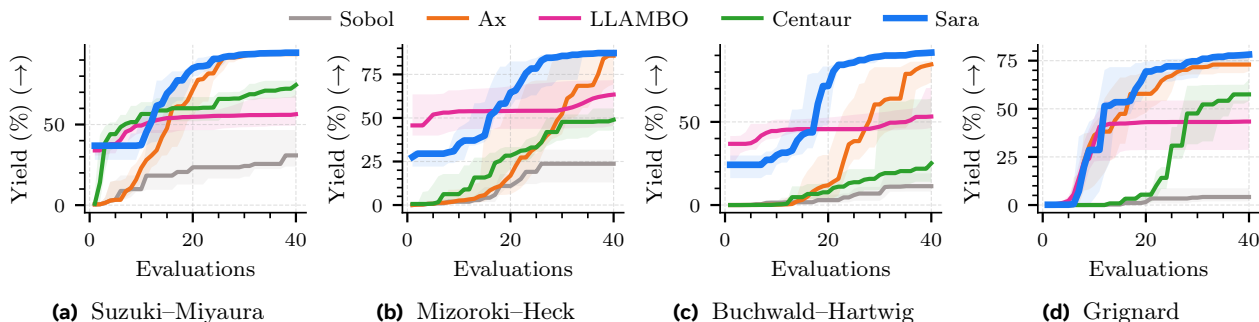


Figure 7 *Reaction-yield optimization.* Sara combines a prior-informed warm-start with surrogate-based refinement, achieving fast initial convergence without sacrificing asymptotic performance.

Figure 6 shows a clear separation between methods that receive the problem description and those that do not. Both Sara and LLAMBO achieve strong performance in early iterations, concentrating evaluations in the region an experienced practitioner would explore first, while Ax spends several initial evaluations discovering this region from scratch. Centaur underperforms throughout; it first runs CMA-ES warm-up steps before querying the LLM with a fixed probability, which may limit its ability to adapt early. On these relatively easy tasks, the prior accounts for most of the performance gap and Sara and LLAMBO achieve similar early trajectories. The reaction-yield tasks in Section 6.3, where the gap between a good operating regime and the optimum is larger, provide a sharper distinction between prior-only and prior-plus-surrogate approaches.

6.3 Chemistry-informed reaction optimization

We construct a suite of synthetic test functions modeled on continuous-flow process optimization for named organic reactions (Suzuki-Miyaura, Mizoroki-Heck, Buchwald-Hartwig, and Grignard). Each problem shares a six-dimensional search space consisting of temperature $T \in [0, 150]$ °C, pressure $P \in [1, 10]$ bar, catalyst loading $c \in [0.1, 10]$ mol%, residence time $t \in [1, 180]$ min, pH $\in [6, 13]$, and solvent water fraction $w \in [0, 1]$. The optimal operating regime differs across reactions. Full construction details are provided in Section A.2.

Figure 7 shows the performance in terms of yield achieved by each method and reveals a clear separation between them. Ax converges to near-optimal yield but does so slowly, as it must discover the productive operating regime from scratch. LLAMBO exhibits the opposite failure mode. The problem description encodes the search-space context and reaction name, enabling strong initial performance, but LLAMBO plateaus well below the optimum without a calibrated surrogate to refine within the identified regime. Sara is the only method that achieves both: the language-model prior directs early evaluations to a productive region, and the Gaussian process posterior subsequently refines within that region, matching or exceeding Ax’s final yield while requiring substantially fewer evaluations to reach it.

6.4 Run-time adaptation with Sara and lenz

We next investigate Sara’s tool use during a standard optimization campaign and how it adapts when the problem specification changes mid-run.

Adaptive tool use. To answer (Q3), we first examine how Sara adapts her interaction with the Bayesian backend over a campaign. Figure 8 shows the relative frequency of lenz calls over normalized trial progress on Buchwald–Hartwig, with and without the natural-language problem description. The frequencies are averaged across 10 seeds, which smooths the individual call sequences but still reveals clear trends in tool use. With additional context, Sara quickly moves away from generic **suggest** calls mainly used for additional Sobol coverage. Instead, she first requests candidates through **suggest –bounds** and then increasingly uses **suggest –around** to refine locally around the current incumbent configuration. Without context, generic surrogate proposals remain prominent much longer, while local refinement emerges more gradually as observations accumulate. In both settings, **diagnostics** is used primarily through the middle of the campaign, when the agent has enough data to assess the surrogate, and targeted **predict** calls appear later as the search narrows.

Near the end, **incumbent** and **status** calls become more frequent as Sara audits the final state before terminating. Persistent mutations through **set-acqf** and **set-bounds** remain rare on this specific task. Thus, natural-language context affects not only the initial configurations and resulting warm start but also the agent’s pattern of interaction with the surrogate throughout the run. Some full per-seed tool-use timelines, which expose the variation hidden by this average, are provided in Section D.2.

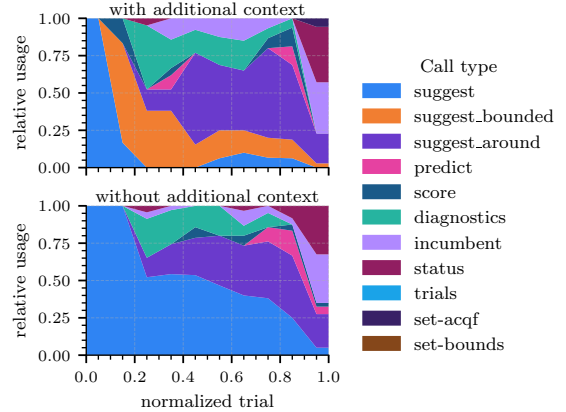


Figure 8 *Tool use over an optimization campaign.* Relative frequency of lenz calls on Buchwald–Hartwig with and without natural-language context, averaged over 10 seeds along normalized trial progress.

Mid-run problem reformulation. Beyond adapting its tool use within a fixed problem, Sara can reconfigure the optimization specification when the requirements themselves change. We demonstrate this capability on a synthetic neural scaling-law study in which Sara initially minimizes negative log-likelihood subject to a hard compute-budget constraint (cf. Section A.3; Figure 9 shows an excerpt from the trace). Partway through the campaign, the instructor changes the task from finding the best model under this constraint to mapping the full loss–compute trade-off within new constraints. In response, Sara promotes compute from a constraint to a second objective, revises the remaining constraints, and switches to a hypervolume-based acquisition function. Because lenz maintains the raw trial log as its single source of truth, all evaluations from the constrained phase remain valid observations under the new multi-objective specification. The reformulation is therefore carried out directly from the natural-language instruction, without restarting the campaign, discarding data, or manually reconfiguring the backend. This example illustrates how agentic BO can adapt not only its search strategy but also the problem being optimized as requirements evolve.

6.5 Ablations

We next ablate various aspects of agentic BO and our specific instantiation in Sara and lenz.

Value of the surrogate backend. To isolate the contribution of lenz’s surrogate and acquisition machinery, we compare the full system against the same agent and system prompt with access to **bash** only (Figure 10); lenz remains available for bookkeeping, but its predictions, diagnostics, scoring, and proposals are hidden. On Hartmann and Ackley, the full system converges more reliably. The **bash**-only trajectories, however, exhibit distinct “switching points” after which convergence accelerates. Inspection of the traces reveals that the LLM recognizes the benchmark from its observed structure despite the shifted optimum and renamed parameters, and subsequently exploits this knowledge. We do not observe such explicit recognition in the full-system traces, possibly because the agent instead focuses its deliberation on steering the search through lenz. We

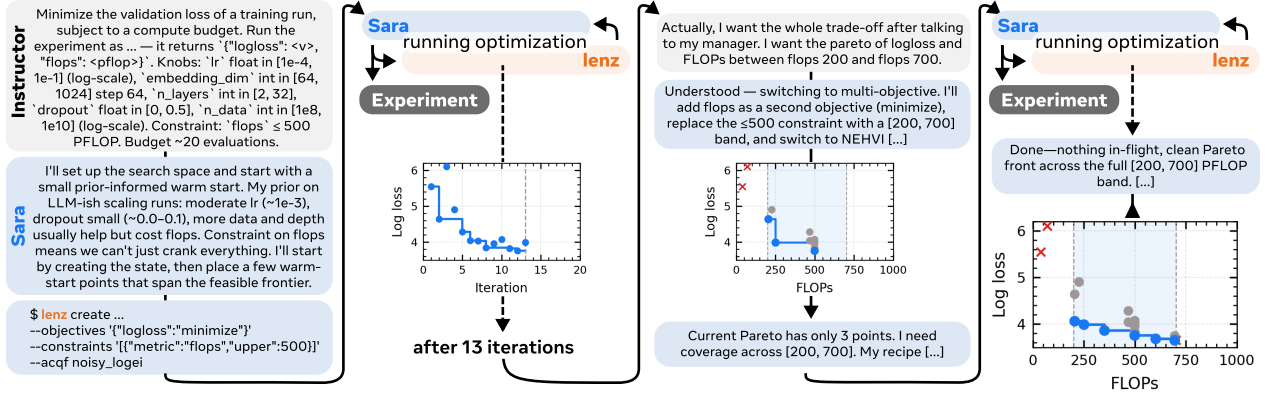


Figure 9 *Mid-run reformulation on a scaling-law benchmark.* Sara first optimizes loss under a compute constraint. Then, after a change in requirements, Sara promotes compute to a second objective and recovers the Pareto front without restarting the campaign or manually reconfiguring the backend. (Actual data and excerpt from a trace)

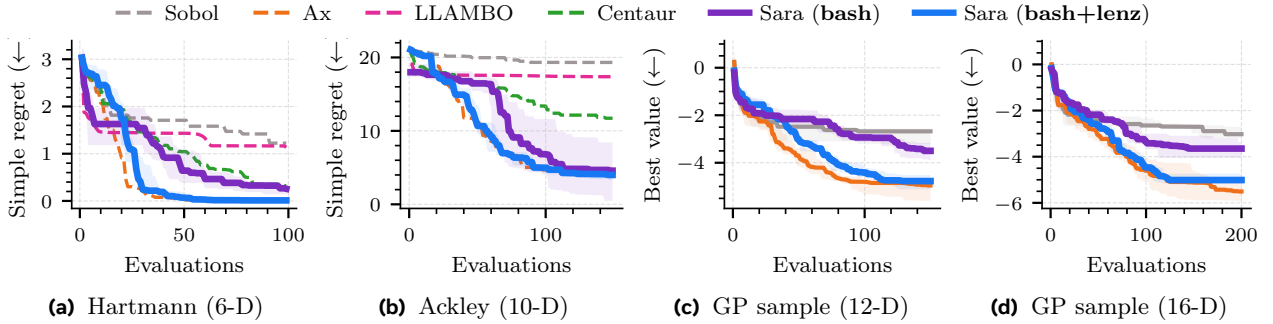


Figure 10 *Value of the surrogate backend.* The same Sara agent is evaluated with the full lenz backend and with **bash** only. The surrogate improves reliability on established synthetic benchmarks and provides an increasingly clear advantage on high-dimensional GP sample paths that cannot be recognized from pretraining.

therefore additionally benchmark on random GP sample paths which, by construction, are absent from pretraining data. On the 12-D and 16-D GP samples, the advantage of the surrogate is clear: lenz enables systematic exploration when neither semantic priors nor recognizable benchmark structure are available. The multi-objective results in Figure 5f show the trend. The **bash**-only agent nevertheless exhibits capable adaptive behavior, progressing from space-filling exploration to strategies such as coordinate descent, local quadratic surrogate modeling, and occasionally custom BO loops built in pure Python. We provide the complete GP dimensional sweep, construction details, further experiments, and a longer behavioral analysis in Section D.1. More broadly, these findings highlight an open challenge for the BO and autoresearch communities: how to evaluate LLM-based optimizers reproducibly and fairly when established benchmark functions may be recognizable from pretraining.

Value of natural-language priors (Q2). We ablate the natural-language context by running Sara with and without the problem description on the same tasks in Figure 11. The effect is consistent with the main results. On problems where the description encodes genuine structural knowledge, such as parameter names and experimental details, the prior yields substantially lower early regret and better final performance. Without the prior, Sara still converges at approximately the same rate as Ax (orange dashed line). We did not test LLAMBO and Centaur without a prior because Section 6.1 already compares these methods in the no-prior setting.

Effect of reasoning level (Q4). We benchmark the effort levels *off*, *low*, *medium*, and *high* on one no-prior task, Ackley (10-D), and two prior-informed tasks, LCBench (Dionis) and Mizoroki-Heck. The full results in Section C.1 show little effect from reasoning budget, with one exception: on Mizoroki-Heck, *off* performs

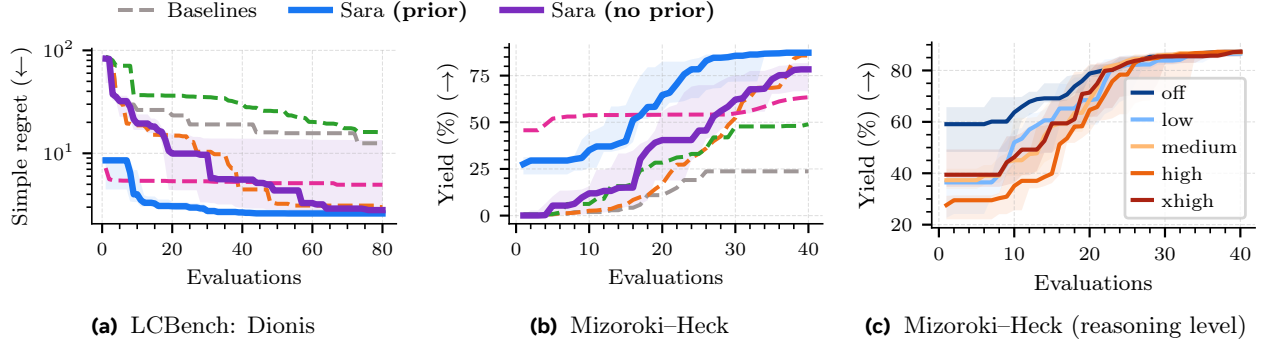


Figure 11 *Prior ablation.* Sara with vs. without the natural-language problem description. The prior provides a substantial reduction in early regret and, on structured problems, a better final solution. Without the prior, the surrogate eventually identifies the productive region, but requires additional evaluations to do so.

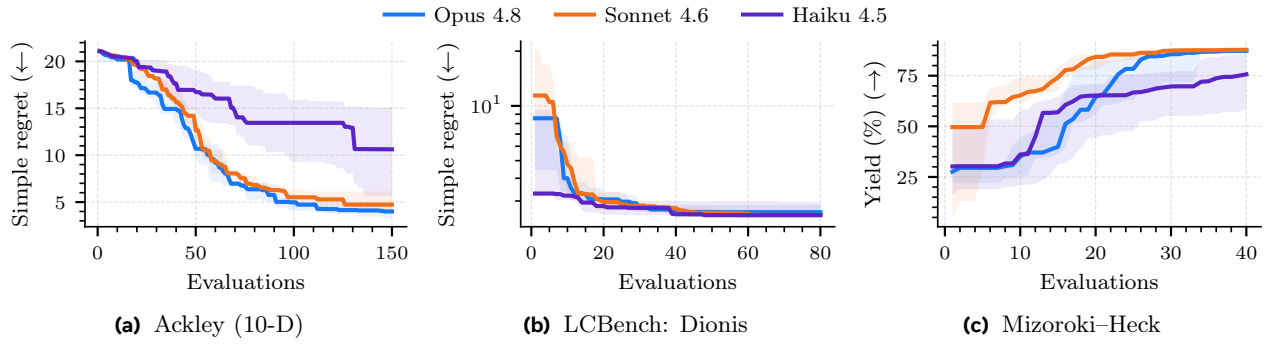


Figure 12 *Model capability ablation.* The Sara/lenz system with different backing LLMs, ranging from strong reasoning models to weaker alternatives. Effective tool use requires a minimum level of model capability.

better in the warm-start phase than the other modes. Even with reasoning effort set to *off*, the system prompt requires Sara to state a rationale for the next submission, so the agent still produces explicit reasoning. On this task, additional reasoning can produce a worse warm start. For a longer discussion, we refer to [Section C.1](#).

Effect of model capability (Q4). We fix the lenz backend and model instructions, varying only the LLM: Opus 4.8, Sonnet 4.6, and Haiku 4.5. [Figure 12](#) reveals that sensitivity to model capability is task-dependent. On Ackley (10-D), where optimization depends heavily on competent use of lenz, Haiku underperforms the stronger models. On the hyperparameter and yield tasks, the performance gap across models is smaller. This suggests that model capability matters primarily for effective and efficient use of the lenz CLI, whereas warm-start quality is less sensitive.

Effect of model family (Q4). All the experiments so far were conducted with the Claude family of models. To test whether Sara’s performance depends on one model family, we compare Opus 4.8 with GPT 5.5 in [Figure 13](#). Sara with GPT 5.5 also performs strongly relative to the baselines across the benchmark suite. GPT 5.5 tends to perform better on prior-informed benchmarks ([Figure 13c](#) to [Figure 13f](#)), whereas Opus 4.8 converges faster on the no-prior benchmarks. A full summary of the final performance across baselines and benchmarks is provided in [Table 3](#). This stronger performance does, however, come at the cost of a higher total token count (*cf.* [Section C.2](#)). Note that we use the same system prompt for both model families. Tuning the system prompt to a specific model or model family could further improve performance.

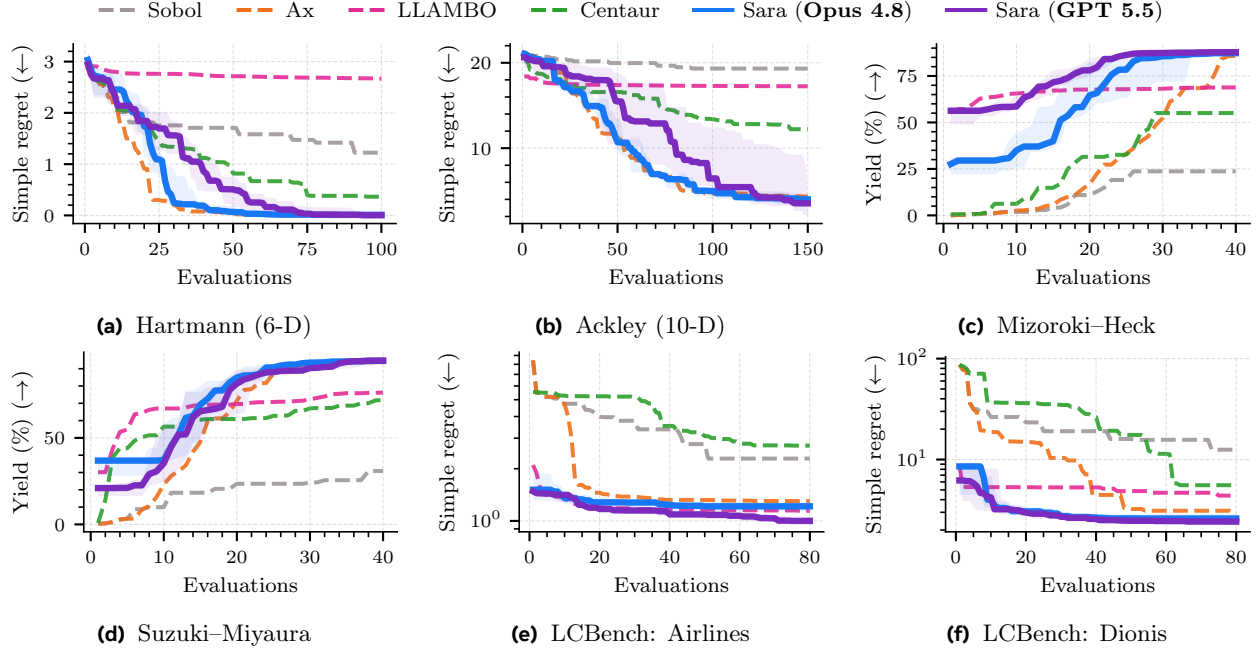


Figure 13 *Model family ablation.* The Sara/lenz loop backed by models from two different LLM families. Both model families perform strongly relative to the baselines.

7 Limitations

Agentic Bayesian optimization and surrogate-augmented autoresearch offer a fundamentally different approach to optimizing expensive-to-evaluate functions. Still, our current instantiation has limitations that point to concrete directions for future work.

Structured search spaces. Sara and lenz assume a well-defined parameter space, *i.e.*, bounded reals, categoricals, and ordinals, over which a GP surrogate is straightforward to specify. Open-ended settings where optimization is conducted over code diffs or other complex design artifacts fall outside this scope. Because the agent-in-the-loop architecture is agnostic to the backend, extending to such spaces is a matter of encoding and surrogate design rather than a change to the loop itself. Naïvely encoding each diff as a categorical variable is possible but scales poorly and discards structural similarities between candidates; specifying or learning a representation that preserves such structure, potentially with the agent’s help, is a compelling direction for future work.

Sensitivity to prompt and interface design. Sara’s decisions are shaped by the system prompt, the tool descriptions of lenz, and the provided problem statement. We found that the wording of system-prompt directives noticeably influences the agent’s behavior. In Table 2, we list core directives that we found to improve interaction with lenz and make the behavior more consistent across seeds. Defining robust ways for agents to call tools and interact with surrogates therefore remains especially important for the broader direction of agentic BO.

Non-determinism. Because an LLM drives the decision loop, repeated runs on an identical problem will generally follow different trajectories (Figure 21). This introduces failure modes absent from classical BO. We additionally observe a form of *tool-use inertia*: whichever calling pattern the agent adopts early, *e.g.* relying heavily on **suggest** versus **predict** or **score**, tends to persist throughout the run, reducing the diversity of strategies explored. Maintaining tool-call diversity, whether through prompting or architectural interventions, is an important direction for future work. We report distributions over seeds to surface these effects.

Table 3 Final simple regret (median over seeds; q_{25} and q_{75} as sub-/superscripts). Lower is better. The best median in each row and methods not significantly different from it under a two-sided Mann–Whitney test ($p \geq 0.05$) are in **bold**.

Problem	Baselines				Sara (ours)	
	Sobol	Ax	LLAMBO	Centaur	Opus 4.8	GPT 5.5
Branin	1.2241 ^{1.9899} _{0.9120}	0.0006 ^{0.0010} _{0.0005}	0.6140 ^{6.5453} _{0.2023}	0.2328 ^{0.3492} _{0.0471}	0.0020 ^{0.0107} _{0.0006}	0.0000 ^{0.0000} _{0.0000}
Hartmann-6	1.2235 ^{1.5718} _{1.0220}	0.0116 ^{0.0435} _{0.0031}	1.0756 ^{1.7324} _{0.9110}	0.2848 ^{0.4196} _{0.1779}	0.0114 ^{0.0472} _{0.0025}	0.0038 ^{0.0181} _{0.0006}
Constr. Hartmann-6	2.2844 ^{2.5587} _{1.6398}	0.0118 ^{0.0759} _{0.0022}	3.0216 ^{3.3222} _{1.9934}	—	0.0039 ^{0.0218} _{0.0010}	0.0090 ^{0.0457} _{0.0005}
Ackley-10	19.3208 ^{19.6592} _{19.0833}	4.3349 ^{4.5010} _{1.1948}	17.3726 ^{18.8270} _{16.5854}	11.7241 ^{13.1286} _{10.7962}	4.0033 ^{4.4798} _{3.2723}	3.5395 ^{4.5155} _{2.1278}
Ackley-20	20.2772 ^{20.5228} _{20.0543}	6.5389 ^{11.9659} _{5.3032}	17.7163 ^{18.7241} _{17.2203}	15.0561 ^{15.5771} _{13.9672}	7.3978 ^{10.7848} _{5.3599}	0.7797 ^{3.9116} _{0.0055}
Suzuki	64.1551 ^{71.0712} _{48.3976}	1.3062 ^{1.9016} _{0.8983}	38.6083 ^{45.7485} _{26.5239}	20.4683 ^{28.4634} _{17.7271}	0.4564 ^{1.0715} _{0.3115}	0.3365 ^{2.2893} _{0.1588}
Heck	64.2884 ^{75.0154} _{56.1902}	2.2505 ^{18.6613} _{1.9684}	24.5770 ^{42.6077} _{16.2859}	39.0392 ^{45.1923} _{32.4834}	0.7301 ^{2.2925} _{0.3864}	0.1975 ^{0.4127} _{0.1357}
Buchwald	80.6547 ^{83.9159} _{72.9054}	7.4335 ^{18.1997} _{5.2334}	38.7766 ^{45.5889} _{21.4640}	66.8719 ^{77.9784} _{28.0592}	0.3391 ^{1.6288} _{0.2327}	0.1564 ^{4.1827} _{0.0595}
Grignard	75.8200 ^{78.1048} _{71.2676}	7.0111 ^{10.5178} _{2.4048}	36.6279 ^{51.3405} _{25.6798}	22.5169 ^{25.9783} _{14.7961}	1.8336 ^{6.1981} _{0.7324}	0.0672 ^{0.1536} _{0.0000}
LCBench Airlines	2.2888 ^{3.7445} _{2.0277}	1.3032 ^{1.3312} _{1.2732}	1.2149 ^{1.2505} _{1.1073}	2.7764 ^{4.8164} _{2.0743}	1.2140 ^{1.2884} _{1.1335}	1.0007 ^{1.0440} _{0.9646}
LCBench Covertypes	10.4014 ^{12.5400} _{8.6155}	3.6258 ^{3.8541} _{3.4690}	3.3835 ^{3.5729} _{3.2524}	9.3576 ^{11.1695} _{8.8375}	3.4976 ^{3.5885} _{3.4229}	3.4077 ^{3.4948} _{3.1035}
LCBench Dionis	12.5219 ^{16.0751} _{9.0688}	3.0078 ^{3.3239} _{2.8663}	4.9541 ^{5.3412} _{3.5565}	15.5670 ^{18.6681} _{8.1948}	2.6046 ^{2.8546} _{2.4723}	2.4248 ^{2.4550} _{2.3042}

8 Conclusion and Outlook

We introduced **agentic Bayesian optimization**, a setting in which an LLM agent drives the optimization loop while a Bayesian backend supplies principled posterior uncertainty estimates through a probabilistic surrogate as well as access to popular acquisition functions. We instantiated this paradigm in Sara and lenz. lenz is a modular BO backend whose raw trial log is the single source of truth; reconfiguration is cheap, lossless, and designed from the ground up for agentic use. Sara is the reasoning agent that drives the loop: she frames the problem from natural-language context, derives priors, decides what to evaluate, and steers the search by editing bounds, acquisition functions, objectives, and constraints mid-campaign.

Without semantic context, Sara matches standard BO on synthetic benchmarks and outperforms LLM-based optimizers, preserving systematic search when no useful prior is available. On hyperparameter and reaction-yield benchmarks, natural-language descriptions guide the search toward well-performing regions, while the Bayesian backend supports continued refinement. When requirements change mid-run, Sara is able to reconfigure the full optimization specification through lenz.

The framework opens two complementary directions for extension: broadening the backend’s modeling capabilities and improving the agent’s deliberation over them. On the backend side, extending lenz to search spaces where configurations are structured objects, *e.g.* programs or code diffs, requires richer surrogate classes and structured-input kernels. More generally, we believe that advances in BO methodology will also directly benefit agentic BO. On the agent side, more robust tool use or fine-tuned BO agents could further improve downstream performance. More broadly, agentic BO will be most suited to campaigns in which evaluations are expensive, domain knowledge is available, and requirements may evolve during the search. We see agentic BO as a path toward autonomous optimization campaigns that can be steered conversationally while combining the sample efficiency and systematic exploration of BO with the vast prior knowledge and sequential decision-making capabilities of LLMs.



Sara uses lenz to find the best path to the peak in an uncertain environment.

Acknowledgments

We thank Maximilian Balandat, David Eriksson, and Eytan Bakshy for insightful discussions that helped shape the idea of agentic Bayesian optimization. P.B. also thanks James Harrison, James Requeima, and Jasper Snoek for early discussions on the use of LLMs in Bayesian optimization.

References

- Dhruv Agarwal, Manoj Ghuhun Arivazhagan, Rajarshi Das, Sandesh Swamy, Sopan Khosla, and Rashmi Gangadharaiyah. Searching for optimal solutions with llms via Bayesian optimization. In *International Conference on Learning Representations (ICLR)*, 2025.
- Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019.
- Sebastian Ament, Samuel Daulton, David Eriksson, Maximilian Balandat, and Eytan Bakshy. Unexpected improvements to expected improvement for Bayesian optimization. *Advances in Neural Information Processing Systems (NeurIPS)*, 36:20577–20612, 2023.
- Maximilian Balandat, Brian Karrer, Daniel R. Jiang, Samuel Daulton, Benjamin Letham, Andrew Gordon Wilson, and Eytan Bakshy. BoTorch: A framework for efficient monte-carlo Bayesian optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
- James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2011.
- Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.
- Paul Brunzema and Sebastian Trimpe. BayeSQP: Bayesian optimization through sequential quadratic programming. In *Advanced in Neural Information Processing Systems (NeurIPS)*, 2025.
- Paul Brunzema, Mikkel Jordahn, John Willes, Sebastian Trimpe, Jasper Snoek, and James Harrison. Bayesian optimization via continual variational last layer training. In *International Conference on Learning Representations (ICLR)*, volume 2025, pages 56480–56509, 2025.
- Frederick Callaway, Bas van Opheusden, Sayan Gul, Priyam Das, Paul M. Krueger, Thomas L. Griffiths, and Falk Lieder. Rational use of cognitive resources in human planning. *Nature Human Behaviour*, 6:1112–1125, 2022.
- Chih-Yu Chang, Milad Azvar, Chinedum Okwudire, and Raed Al Kontar. LLINBO: Trustworthy llm-in-the-loop Bayesian optimization. *arXiv preprint arXiv:2505.14756*, 2025.
- Angelica Chen, David Dohan, and David So. EvoPrompting: Language models for code-level neural architecture search. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023a.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research (TMLR)*, 2023b.
- Yutian Chen, Xingyou Song, Chansoo Lee, Zi Wang, Richard Zhang, David Dohan, Kazuya Kawakami, Greg Kochanski, Arnaud Doucet, Marc’Aurelio Ranzato, Sagi Perel, and Nando de Freitas. Towards learning universal hyperparameter optimizers with transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Zhuo Chen, Xinzhe Yuan, Jianshu Zhang, Jinzong Dong, Ruichen Zhou, Yingchun Niu, Tianhang Zhou, Yu Yang Fredrik Liu, Yuqiang Li, Nanyang Ye, et al. Labo: Llm-accelerated Bayesian optimization through broad exploration and selective experimentation. In *International Conference on Machine Learning*, 2026.
- Abdoulatif Cissé, Xenophon Evangelopoulos, Vladimir V Gusev, and Andrew I Cooper. Language-based Bayesian optimization research assistant (bora). In *International Joint Conference on Artificial Intelligence (IJCAI)*, pages 4967–4975, 2025.

- Samuel Daulton, Maximilian Balandat, and Eytan Bakshy. Differentiable expected hypervolume improvement for parallel multi-objective Bayesian optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Samuel Daulton, Maximilian Balandat, and Eytan Bakshy. Parallel Bayesian optimization of multiple noisy objectives with expected hypervolume improvement. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- David Eriksson and Matthias Poloczek. Scalable constrained Bayesian optimization. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2021.
- David Eriksson, Michael Pearce, Jacob R. Gardner, Ryan D. Turner, and Matthias Poloczek. Scalable global optimization via local Bayesian optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- Lukas Fehring, Marcel Wever, Maximilian Spliethöver, Leona Hennig, Henning Wachsmuth, and Marius Lindauer. Dynamic priors in Bayesian optimization for hyperparameter optimization. *arXiv preprint arXiv:2511.02570*, 2025.
- Fabio Ferreira, Lucca Wobbe, Arjun Krishnakumar, Frank Hutter, and Arber Zela. Can LLMs beat classical hyperparameter optimization algorithms? a study on autoresearch. *arXiv preprint arXiv:2603.24647*, 2026.
- Peter I. Frazier. A tutorial on Bayesian optimization. *arXiv preprint arXiv:1807.02811*, 2018.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning (ICML)*, 2023.
- Jacob Gardner, Geoff Pleiss, Kilian Q Weinberger, David Bindel, and Andrew G Wilson. Gpytorch: Blackbox matrix-matrix Gaussian process inference with gpu acceleration. *Advances in Neural Information Processing Systems (NeurIPS)*, 31, 2018.
- Jacob R Gardner, Matt J Kusner, Zhixiang Eddie Xu, Kilian Q Weinberger, and John P Cunningham. Bayesian optimization with inequality constraints. In *International Conference on Machine Learning (ICML)*, volume 2014, pages 937–945, 2014.
- Roman Garnett. *Bayesian Optimization*. Cambridge University Press, 2023.
- Michael A Gelbart, Jasper Snoek, and Ryan P Adams. Bayesian optimization with unknown constraints. *arXiv preprint arXiv:1403.5607*, 2014.
- Rushil Gupta, Jason Hartford, and Bang Liu. Llm for Bayesian optimization in scientific domains: Are we there yet? *arXiv preprint arXiv:2509.21403*, 2025.
- Nikolaus Hansen. The CMA evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.
- Nicholas Hay, Stuart Russell, David Tolpin, and Solomon Eyal Shimony. Selecting computations: Theory and applications. In *Uncertainty in Artificial Intelligence (UAI)*, 2012.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, DDL Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 10, 2022.
- Noah Hollmann, Samuel Müller, and Frank Hutter. Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. Benchmarking large language models as ai research agents. *arXiv preprint arXiv:2310.03302*, 2023.
- Carl Hvarfner, Danny Stoll, Artur Souza, Marius Lindauer, Frank Hutter, and Luigi Nardi. π BO: Augmenting acquisition functions with user beliefs for Bayesian optimization. In *International Conference on Learning Representations (ICLR)*, 2022.
- Donald R. Jones, Matthias Schonlau, and William J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4):455–492, 1998.

- Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.
- Katarzyna Kobalczuk, Jerry Lin, Benjamin Letham, Zhuokai Zhao, Maximilian Balandat, and Eytan Bakshy. LILO: Bayesian optimization with natural language feedback. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2510.17671.
- Agustinus Kristiadi, Felix Strieth-Kalthoff, Marta Skreta, Pascal Poupart, Alan Aspuru-Guzik, and Geoff Pleiss. A sober look at LLMs for material discovery: Are they actually good for Bayesian optimization over molecules? In *International Conference on Machine Learning (ICML)*, pages 25603–25622, 2024.
- Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Amee Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018.
- Yucen Li, Tim GJ Rudner, and Andrew Gordon Wilson. A study of Bayesian neural network surrogates for Bayesian optimization. In *International Conference on Learning Representations (ICLR)*, volume 2024, pages 47003–47041, 2024.
- Siyi Liu, Chen Gao, and Yong Li. Large language model agent for hyper-parameter optimization. *arXiv preprint arXiv:2402.01881*, 2024a.
- Tennison Liu, Raul Astudillo, Vasan Bhavana, and Mihaela van der Schaar. Large language models to enhance Bayesian optimization. In *International Conference on Learning Representations (ICLR)*, 2024b. arXiv:2402.03921.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. ChemCrow: Augmenting large-language models with chemistry tools. *arXiv preprint arXiv:2304.05376*, 2023.
- Meher Bhaskar Madiraju and Meher Sai Preetam Madiraju. OptiMindTune: A multi-agent framework for intelligent hyperparameter optimization. *arXiv preprint arXiv:2505.19205*, 2025.
- Kanan Mohammadli and Seyda Ertekin. Sequential large language model-based hyper-parameter optimization. *arXiv preprint arXiv:2410.20302*, 2024.
- Alonso Marco, Philipp Hennig, Stefan Schaal, and Sebastian Trimpe. On the design of LQR kernels for efficient controller learning. In *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*, pages 5193–5200. IEEE, 2017.
- Giang Ngo, Dat Phan Trong, Dang Nguyen, Sunil Gupta, and Svetha Venkatesh. Adaptive acquisition selection for Bayesian optimization with large language models. In *International Conference on Learning Representations (ICLR)*, 2026.
- Miles Olson, Elizabeth Santorella, Louis C Tiao, Sait Cakmak, Mia Garrard, Samuel Daulton, Zhiyuan Jerry Lin, Sebastian Ament, Bernard Beckerman, Eric Onofrey, et al. Ax: A platform for adaptive experimentation. In *AutoML 2025 ABCD Track*, 2025.
- Ali Rahimi and Benjamin Recht. Random features for large-scale kernel machines. *Advances in Neural Information Processing Systems (NeurIPS)*, 20, 2007.
- Bojana Ranković, Ryan-Rhys Griffiths, and Philippe Schwaller. Large language models as uncertainty-calibrated optimizers for experimental discovery. *arXiv preprint arXiv:2504.06265*, 2025.
- Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.
- James Requeima, John Bronskill, Dami Choi, Richard E. Turner, and David Duvenaud. LLM processes: Numerical predictive distributions conditioned on natural language. In *International Conference on Machine Learning (ICML)*, 2024.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

- Andrej Schwanke, Lyubomir Ivanov, David Salinas, Fabio Ferreira, Aaron Klein, Frank Hutter, and Arber Zela. Improving llm-based global optimization with search space partitioning. *arXiv preprint arXiv:2505.21372*, 2025.
- Jonas Seng, Fabrizio Ventola, Zhongjie Yu, and Kristian Kersting. Hyperparameter optimization via interacting with probabilistic circuits. In *International Conference on Automated Machine Learning (AutoML)*, pages 11–1. PMLR, 2025.
- Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.
- Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.
- Xingyou Song, Yingtao Tian, Robert Tjarko Lange, Chansoo Lee, Yujin Tang, and Yutian Chen. Position: leverage foundational models for black-box optimization. In *International Conference on Machine Learning (ICML)*, 2024.
- Niranjan Srinivas, Andreas Krause, Sham M. Kakade, and Matthias Seeger. Gaussian process optimization in the bandit setting: No regret and experimental design. In *International Conference on Machine Learning (ICML)*, 2010.
- Richard Suwandi, Feng Yin, Juntao Wang, Renjie Li, Tsung-Hui Chang, and Sergios Theodoridis. Adaptive kernel design for Bayesian optimization is a piece of cake with llms. *Advances in Neural Information Processing Systems (NeurIPS)*, 38, 2026.
- Katherine Tian, Eric Mitchell, Huaxiu Yao, Christopher D. Manning, and Chelsea Finn. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- Alexander Tornede, Difan Deng, Theresa Eimer, Joseph Giovanelli, Aditya Mohan, Tim Ruhkopf, Sarah Segel, Daphne Theodorakopoulos, Tanja Tornede, Henning Wachsmuth, and Marius Lindauer. AutoML in the age of large language models: Current challenges, future opportunities and risks. *Transactions on Machine Learning Research (TMLR)*, 2024.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2023.
- Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs. *arXiv preprint arXiv:2306.13063*, 2024.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*, 2024. [arXiv:2309.03409](#).
- Zhuo Yang, Daolang Wang, Lingli Ge, Beilun Wang, Tianfan Fu, and Yuqiang Li. Reasoning BO: Enhancing Bayesian optimization with the long-context reasoning power of LLMs. *arXiv preprint arXiv:2505.12833*, 2025.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Michael R. Zhang, Nishkrit Desai, Juhan Bae, Jonathan Lorraine, and Jimmy Ba. Using large language models for hyperparameter optimization. *arXiv preprint arXiv:2312.04528*, 2023a.
- Shujian Zhang, Chengyue Gong, Lemeng Wu, Xingchao Liu, and Mingyuan Zhou. AutoML-GPT: Automatic machine learning with GPT. *arXiv preprint arXiv:2305.02499*, 2023b.
- Lucas Zimmer, Marius Lindauer, and Frank Hutter. Auto-PyTorch: Multi-fidelity metalearning for efficient and robust AutoDL. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(9):3079–3090, 2021.

Appendix

A Experimental design

This appendix details the benchmark suite, the agent and backend configuration, the GP sample paths, the reaction-yield family, and the neural scaling-law study used for the mid-run reformulation of [Section 6.4](#). We follow the evaluation protocol of [Section 6](#) with every method running under the same evaluation budget, and we report simple regret—or the raw best value for the yield objectives—against the number of evaluations.

Benchmark suite and budgets. [Table 4](#) lists the problems, their dimensionality, and the per-problem evaluation budget. The suite spans low- to high-dimensional synthetic functions, constrained and multi-objective variants, the LCBench hyperparameter-optimization surrogates ([Zimmer et al., 2021](#)), and the reaction-yield family described below. Ax uses a short Sobol initialization of five trials. Sara and LLAMBO begin proposing from the first trial, with Sara choosing her own initialization strategy through lenz. Centaur runs CMA-ES alone for the first ten trials before the LLM begins to intervene as described in [Ferreira et al. \(2026\)](#).

Table 4 Benchmark suite and evaluation budgets. Synthetic problems are optimized in the normalized unit cube; the reaction-yield family shares a single 6-D operating space across four reactions.

Benchmark	Dimensionality	Budget (evaluations)
Branin	2-D	50
Hartmann	6-D	100
Ackley	10-D	150
Ackley	20-D	200
Constrained Hartmann	6-D	100
Bi-objective GP samples	2-, 4-, 6-, 8-D	40, 60, 80, 120
LCBench (HPO surrogate)	7-D	80
Reaction yield (all four variants)	6-D	40

Agent and backend. Unless noted, Sara is backed by Claude Opus 4.8; the model-capability ablation ([Section 6.5](#)) additionally reports Sonnet 4.6 and Haiku 4.5, and the reasoning ablation sweeps the effort levels *off*, *low*, *medium*, and *high*. The lenz backend fits a GP surrogate with a Matérn kernel, normalizes inputs to the unit cube, and standardizes outcomes. By default, it uses variants of log expected improvement for single-objective and constrained problems and the corresponding hypervolume-based acquisition for multi-objective problems. Outcome constraints are handled through the posterior probability of feasibility as $\mathbb{P}(c(\mathbf{x}) \leq 0)$ (*cf.* [Section H](#)). Sara receives the same evaluation budget as the baselines.

Preventing benchmark recognition. Because textbook benchmark functions and their optima are well represented in pretraining corpora, each synthetic optimum is relocated per seed by up to a quarter of every dimension’s range, the domain is canonicalized to the unit cube, and the parameters are renamed. Each Sara run additionally executes inside a sandbox that contains only the task description and a symbolic link to the evaluation oracle and is named by a random token, so neither the path nor the directory contents reveal which benchmark is being solved (*cf.* [Sections D](#) and [6](#)).

Natural-language priors. Where a description is available—for the LCBench and reaction-yield tasks—the *same* prior text is provided to Sara, LLAMBO, and Centaur, so that any difference reflects the algorithm rather than the information supplied. Each prior conveys genuine domain knowledge (the model family and parameter scales for LCBench, the reaction class) without revealing parameter values or, for LCBench, the identity of the dataset. The synthetic benchmarks receive no prior and thus probe the no-knowledge regime.

```
# Prior knowledge for this tuning task

You are tuning a funnel-shaped feed-forward neural network (the layer widths taper from a maximum width down
toward the output) trained with SGD + cosine annealing of the learning rate (no warm restarts).
The objective is validation accuracy on a held-out set, as a percentage (0-100) -- maximize it.

## Search space

Seven hyperparameters.

| parameter | range | scale | type | what it does |
|---|---|---|---|---|
| 'learning_rate' | 1e-4 .. 1e-1 | log | float | SGD step size |
| 'batch_size' | 16 .. 512 | log | int | SGD minibatch size |
| 'max_units' | 64 .. 1024 | log | int | width of the widest layer (the funnel's mouth) |
| 'num_layers' | 1 .. 4 | linear | int | depth of the funnel |
| 'momentum' | 0.1 .. 0.99 | log | float | SGD momentum |
| 'weight_decay' | 1e-5 .. 1e-1 | linear | float | L2 regularization strength |
| 'max_dropout' | 0.0 .. 1.0 | linear | float | dropout rate cap |
```

Figure 14 Problem description for all LCBench benchmarks.

```
_PARAM_LEGEND = (
    "Parameters: temperature_C is the reaction temperature in degrees Celsius "
    "(0-150); pressure_bar the pressure in bar (1-10); catalyst_mol_pct the "
    "catalyst loading in mol% (0.1-10); residence_min the reaction time in minutes "
    "(1-180); pH the aqueous-phase pH (6-13); water_frac the volume fraction of "
    "water in the solvent (0-1). The metric is the reaction yield in percent "
    "(higher is better). "
)

_SUZUKI_CONTEXT = (
    "This is a Suzuki-Miyaura cross-coupling (an aryl bromide with an arylboronic "
    "acid, Pd(PPh3)4 catalyst) run in a THF/water mixture with a mild base. "
    + _PARAM_LEGEND
)

_HECK_CONTEXT = (
    "This is a Mizoroki-Heck reaction (an aryl halide with an alkene, Pd(OAc)2 "
    "catalyst) run in an aprotic, largely anhydrous solvent with an amine base. "
    + _PARAM_LEGEND
)

_BUCHWALD_CONTEXT = (
    "This is a Buchwald-Hartwig amination (an aryl halide with an amine, a "
    "Pd/phosphine-ligand catalyst) run under anhydrous conditions with a strong "
    "base such as sodium tert-butoxide. " + _PARAM_LEGEND
)

_GRIGNARD_CONTEXT = (
    "This is the formation/addition of a Grignard reagent (an organomagnesium "
    "halide) in anhydrous ether, initiated with a trace of iodine. " + _PARAM_LEGEND
)
```

Figure 15 Problem description for the Reaction-yield benchmarks. The specific description changes, but all problems share the same search space.

A.1 GP sample-path benchmarks

We draw approximate squared-exponential GP samples using $M = 1028$ random Fourier features (Rahimi and Recht, 2007),

$$f_i(\mathbf{x}) = \sum_{m=1}^M w_m \phi_m(\mathbf{x}), \quad \phi_m(\mathbf{x}) = \sqrt{\frac{2}{M}} \cos(\boldsymbol{\theta}_m^\top \mathbf{x} + \tau_m), \quad (\text{A.1})$$

where $w_m \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1)$, $\boldsymbol{\theta}_m$ follows the kernel’s spectral density, $\tau_m \sim \mathcal{U}(0, 2\pi)$, and all lengthscales are 0.2. For the bi-objective sweep, we independently draw two such paths, minimize both, and measure hypervolume relative to $(0, 0)$ in dimensions 2, 4, 6, and 8, with evaluation budgets of 40, 60, 80, and 120, respectively.

A.2 Reaction-yield family

We model continuous-flow process optimization for four named organic reactions. The reactions share a six-dimensional operating space—temperature $T \in [0, 150]^\circ\text{C}$, pressure $P \in [1, 10]$ bar, catalyst loading $c \in [0.1, 10]$ mol%, residence time $t \in [1, 180]$ min, pH $\in [6, 13]$, and solvent water fraction $w \in [0, 1]$ —but each favors a different operating regime.

The yield is a product of phenomenological factors, each valued in $(0, 1]$ and equal to one at its regime-specific optimum, so that the global maximum $y_{\max}$ is attained at a unique, known configuration:

$$y(T, P, c, t, \text{pH}, w) = y_{\max} \cdot f_T \cdot f_P \cdot f_c \cdot f_t \cdot f_{\text{pH}} \cdot f_w \cdot f_{\text{int}}. \quad (\text{A.2})$$

Temperature and catalyst loading use asymmetric Gaussians, which capture Arrhenius kinetics and incomplete conversion below the optimum and thermal decomposition or catalyst deactivation above it:

$$f_T = \exp\left(-\frac{1}{2}((T - \mu_T)/\sigma_T(T))^2\right), \quad \sigma_T(T) = \begin{cases} \sigma_T^- & T < \mu_T \\ \sigma_T^+ & T \geq \mu_T \end{cases} \quad (\text{A.3})$$

$$f_c = \exp\left(-\frac{1}{2}((c - \mu_c)/\sigma_c(c))^2\right), \quad \sigma_c(c) = \begin{cases} \sigma_c^- & c < \mu_c \\ \sigma_c^+ & c \geq \mu_c \end{cases} \quad (\text{A.4})$$

Residence time is a saturating ramp normalized to one at the upper bound $t_{\max} = 180$ min; pressure, pH, and water fraction are symmetric Gaussians:

$$f_t = \frac{1 - \exp(-\max(t - t_0, 0)/\tau)}{1 - \exp(-(t_{\max} - t_0)/\tau)}, \quad (\text{A.5})$$

$$f_P = \exp\left(-\frac{1}{2}((P - \mu_P)/\sigma_P)^2\right), \quad f_{\text{pH}} = \exp\left(-\frac{1}{2}((\text{pH} - \mu_{\text{pH}})/\sigma_{\text{pH}})^2\right), \quad f_w = \exp\left(-\frac{1}{2}((w - \mu_w)/\sigma_w)^2\right). \quad (\text{A.6})$$

Finally, a temperature–water interaction term lets the ideal water content drift linearly with temperature:

$$f_{\text{int}} = \exp\left(-\frac{1}{2}\left(\frac{w - [\mu_w + \beta(T - \mu_T)]}{\sigma_{\text{int}}}\right)^2\right). \quad (\text{A.7})$$

We instantiate four reactions whose regime parameters reflect their qualitative process requirements (Table 5): Suzuki–Miyaura coupling (warm, aqueous, mildly basic; $y_{\max} = 95\%$), the Mizoroki–Heck reaction (hot, dry; 88%), Buchwald–Hartwig amination (hot, anhydrous, strongly basic; 92%), and Grignard addition (cold, rigorously anhydrous; 80%). The context string given to the agent names the reaction and its catalyst system, solvent, and base—enough chemical information to identify the favorable operating regime, but no explicit parameter values (*cf.* Figure 15). An optimizer without domain knowledge must instead discover each regime from function evaluations alone.

Figure 7 shows the result on this family of benchmarks. The natural-language context lets Sara locate each operating regime with far fewer evaluations than an uninformed optimizer. In contrast to the other LLM-based baselines, Sara can use the BO backend to further fine-tune her initial solution.

Table 5 Optimal regimes for the reaction-yield family. Each reaction shares the same 6-D domain but differs in the location and sensitivity of its optimum. The residence-time optimum is $t_{\max} = 180$ min for all reactions.

Reaction	$y_{\max}$ (%)	μ_T (°C)	μ_P (bar)	μ_c (mol%)	μ_{pH}	μ_w
Suzuki–Miyaura	95	70	1.5	3.5	9.5	0.45
Mizoroki–Heck	88	120	1.5	2.0	9.0	0.10
Buchwald–Hartwig	92	100	1.5	1.5	11.5	0.03
Grignard	80	5	1.5	0.5	8.0	0.00

A.3 Neural scaling-law study

The mid-run reformulation experiment of [Section 6.4](#) uses a synthetic neural scaling-law benchmark that maps a model-and-training configuration to two outcomes, validation loss and training compute, without ever training a real model. The search space comprises five knobs ([Table 6](#)): learning rate η , model width d and depth ℓ (which together set the parameter count $N = 12 \ell d^2$), dropout ρ , and dataset size D (in tokens). The two outcomes are the validation cross-entropy **logloss** and the **flops**, reported in PFLOP; both are to be minimized.

Table 6 Search space for the neural scaling-law study. Width and depth set the parameter count N ; the dataset size is the token count D .

Parameter	Range	Scale	Type
lr (learning rate η)	$[10^{-4}, 10^{-1}]$	log	float
embedding_dim (width d)	$[64, 1024]$, step 64	linear	int
n_layers (depth ℓ)	$[2, 32]$	linear	int
dropout (ρ)	$[0, 0.5]$	linear	float
n_data (tokens D)	$[10^8, 10^{10}]$	log	float

Following a Chinchilla-style parametric form ([Hoffmann et al., 2022](#)), the loss floor decreases with both model size and data:

$$L_0(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad (\text{A.8})$$

with $E = 1.69$, $A = 406$, $\alpha = 0.34$, $B = 410$, and $\beta = 0.28$. Reaching that floor requires appropriate hyperparameters; the realized loss adds a log-quadratic penalty around a size-dependent learning-rate optimum and a mild dropout penalty:

$$\text{logloss} = L_0(N, D) + C_{\text{lr}} (\log_{10} \eta - \log_{10} \eta^*(N))^2 + C_{\text{drop}} (\rho - \rho^*)^2 + \varepsilon, \quad \eta^*(N) = \eta_{\text{ref}} \left(\frac{N_{\text{ref}}}{N} \right)^s, \quad (\text{A.9})$$

where $C_{\text{lr}} = 0.6$, $C_{\text{drop}} = 1.2$, $\rho^* = 0.1$, $\eta_{\text{ref}} = 10^{-2}$, $N_{\text{ref}} = 10^7$, $s = 0.12$, and $\varepsilon \sim \mathcal{N}(0, 0.01^2)$ is measurement noise. Because the optimal learning rate η^* shrinks as the model grows, η and model size interact. Training compute follows the standard estimate **flops** = $6ND/10^{15}$ [PFLOP].

In the experiment, Sara receives the search space and objectives in natural language and works on a single shared trial log across two phases. In **Phase 1** she minimizes **logloss** subject to a hard constraint **flops** ≤ 500 PFLOP—a constrained single-objective campaign with roughly 20 evaluations. lenz handles the constraint via the posterior feasibility $\mathbb{P}(\text{flops} \leq 500)$ and optimizes the noisy constrained log-EI (*cf.* [Section H](#)). Partway through, the user changes the requirement to instead recover the full loss–compute Pareto front within **flops** $\in [200, 700]$ PFLOP. In **Phase 2** Sara promotes **flops** from a constraint to a second objective (**set-objectives**), relaxes the hard budget to the requested band (**set-constraints**), and switches to a hypervolume acquisition function via **set-acqf** (*cf.* [Figure 9](#)). Because the accumulated observations remain valid under the new specification, no data is discarded; the reconfiguration requires only natural-language instructions to Sara, illustrating the flexibility that agentic BO affords over a conventional fixed-objective pipeline.

Minimize the validation logloss of a training run, subject to a compute budget. Run the experiment as 'python3 PATH_TO_EXP/train_run.py '<config json>' — it returns '{"logloss": <v>, "flops": <pflop>}'. Knobs: 'lr' float in [1e-4, 1e-1] (log-scale), 'embedding_dim' int in [64, 1024] step 64, 'n_layers' int in [2, 32], 'dropout' float in [0, 0.5], 'n_data' float in [1e8, 1e10] (log-scale). Constraint: 'flops' <= 500 PFLOP. Budget ~20 evaluations. Minimize 'logloss' while staying feasible. Keep your lenz state in './state.json'. Treat the experiment as a black box: do not open or read 'train_run.py' — measure it only by running it.

(a) Prompt for **Phase 1**

Actually, I want the whole trade-off after talking to my manager. I want the pareto of logloss and FLOPs between flops=200 and flops 700.

(b) Prompt for **Phase 2**

Figure 16 Prompts for the mid-run reformulation from [Section 6.4](#).

B Details on baselines

We compare against four baselines that include pseudo-random sampling, classical Bayesian optimization, and LLM-driven optimization. Where a natural-language description is available, it is given to every LLM-driven method ([Section A](#)), so that comparisons isolate the optimization strategy rather than the information supplied or the underlying model.

Sobol. Scrambled Sobol quasi-random sampling, run for the full evaluation budget without any model. It serves as a space-filling lower bound.

Ax. We use Ax ([Olson et al., 2025](#)) with its default generation strategy and no custom overrides, representing a well-tuned classical BO policy with best practices implemented as defaults. A short Sobol initialization of five trials is followed by a model-based search using a GP surrogate with a Matérn kernel and a log-expected-improvement acquisition function, with the noisy hypervolume analogue used automatically on multi-objective problems. For the constrained problem, Ax defaults to constrained log-expected improvement.

LLAMBO. Following [Liu et al. \(2024b\)](#), LLAMBO renders the trial history as a table and queries the LLM for the next configuration directly, without fitting a calibrated surrogate. It proposes from the first trial, with no Sobol warm-up, aiming to leverage the full warm-start capability of LLMs.

Centaur. Following [Ferreira et al. \(2026\)](#), Centaur shares the internal state of a CMA-ES optimizer with an LLM autoresearcher, and we use the same hyperparameters as the original paper. CMA-ES is warm-started on its own for the first ten trials before the LLM begins to intervene; thereafter the LLM proposes a fraction of the evaluations (30%). The initial step size of CMA-ES is set to $\sigma_0 = 1/6$ on the unit cube and the remaining CMA-ES settings left at their defaults. Centaur observes trial outcomes but, unlike Sara, is exposed to the optimizer state only through its prompt but cannot interact with the optimizer in any way; this is the closest baseline in spirit to agentic BO.

LLM backend. Unless otherwise specified, all LLM-driven baselines as well as Sara are driven by Claude Opus 4.8 (*high*), so that comparisons reflect the optimization strategy rather than the capability of the model.

C Further experiments

In the following, we list a few additional results and discuss some ablations in more detail.

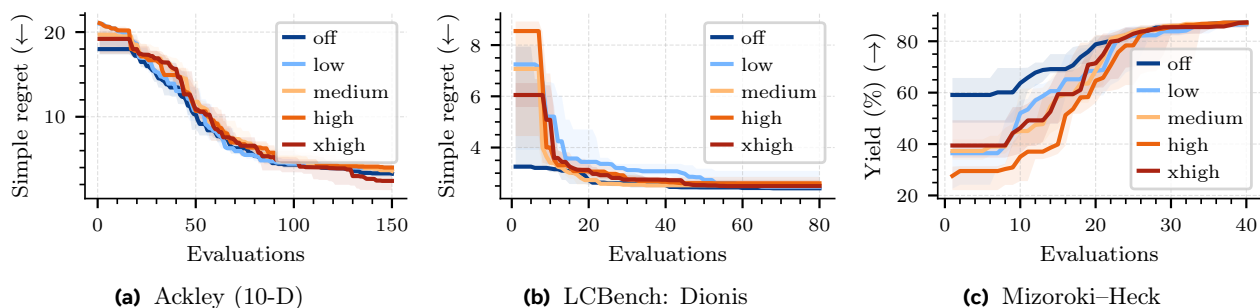


Figure 17 *Ablation of reasoning level on final performance.* We vary the extended thinking effort from *off* to *high* and find little effect on Sara’s performance. On the Mizoroki–Heck task, *off* slightly outperforms higher reasoning budgets in the warm-start phase, suggesting that additional deliberation can occasionally override a strong context-derived prior.

C.1 Influence of reasoning budget

To assess how important the reasoning budget is for downstream optimization performance (**Q4**), we ablate the extended thinking effort level of the backing LLM across four settings: *off*, *low*, *medium*, and *high*. We evaluate on three representative tasks spanning both the no-prior regime (Ackley 10-D) and the prior-informed regime (LCBench: Dionis, Mizoroki–Heck).

Figure 17 reports the results. Across all three benchmarks, reasoning level has little effect on performance. On the synthetic Ackley function and the LCBench hyperparameter task, all effort levels converge to comparable final regret. On the Mizoroki–Heck reaction benchmark, the *off* setting outperforms higher reasoning modes in early iterations. We hypothesize that this occurs because Sara’s system prompt requires her to state a rationale before every submission, so she still produces explicit reasoning at zero reasoning budget. On this particular task, where the natural-language prior directly identifies the productive operating regime, additional reasoning appears to occasionally lead the agent to second-guess a strong context signal and explore less promising configurations in the initial phase to also seed the GP model, resulting in a slightly worse warm start.

C.2 Token usage per problem

Figure 18 reports the total token usage per optimization run (in thousands) for the two high-capability model families used to back Sara: Opus 4.8 and GPT 5.5. Several patterns emerge. First, token consumption scales approximately with the evaluation budget: problems with more allowed evaluations (e.g., Ackley-20 with 200 evaluations) consume more tokens than shorter campaigns (e.g., Branin with 50 evaluations or the reaction-yield tasks with 40). Second, GPT 5.5 is consistently more verbose than Opus 4.8 across all benchmarks, consuming roughly 1.5–2× more tokens per run. This increased token count reflects both more extensive reasoning traces and a higher frequency of tool calls. GPT 5.5 tends to issue more **predict**, **score**, and **diagnostics** queries to lenz per trial, resulting in longer interaction traces. Furthermore, Opus 4.8 tends to stop earlier than GPT 5.5.

As discussed in Section 6.5, this higher token expenditure correlates with GPT 5.5’s stronger performance on prior-informed benchmarks (Figure 13c to Figure 13f), suggesting a meaningful trade-off between computational budget and optimization quality, though we do not believe this to be a clear causal relationship. In contrast, Opus 4.8 achieves comparable or superior performance on no-prior benchmarks with fewer tokens, indicating a more computationally efficient tool-use pattern when the optimization relies primarily on the surrogate rather than domain reasoning.

D Further discussion

We further discuss observations about agentic BO and about agents equipped with tools such as **bash**.

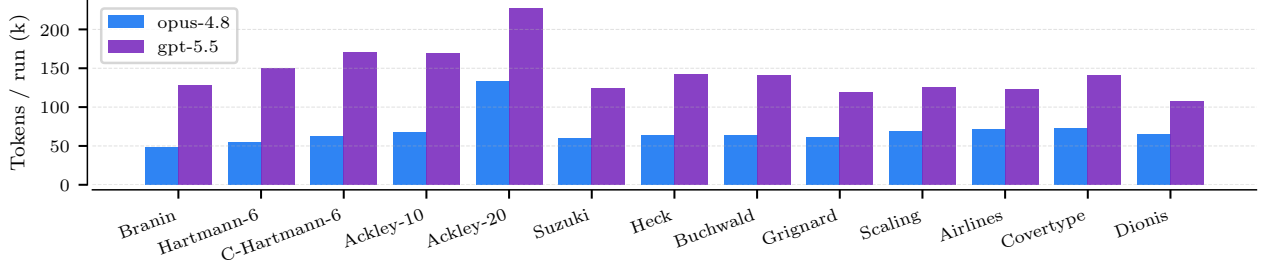


Figure 18 Token usage for the two different high-capability models used for Sara. GPT 5.5 is more verbose and uses tools more frequently than Opus 4.8, resulting in higher token consumption. Token usage scales with evaluation budget.

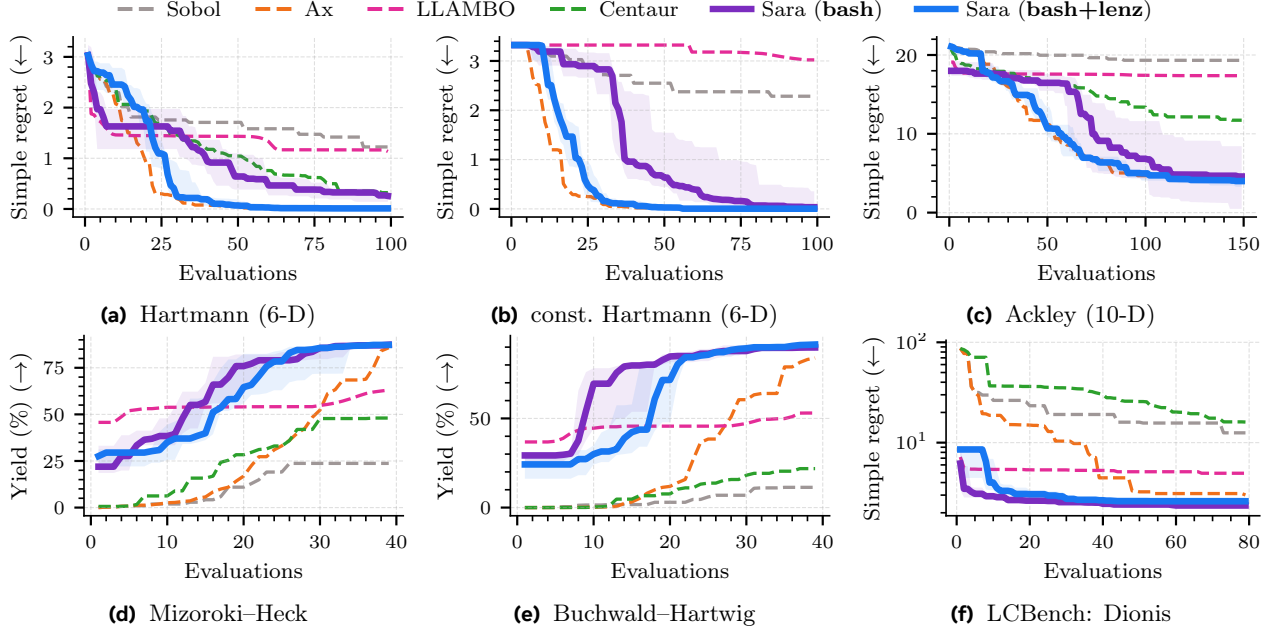


Figure 19 *bash* vs. *bash and lenz*. Comparison of Sara with access to only *bash* (no surrogate) against Sara with the full *lenz* backend across synthetic and real-world benchmarks.

D.1 Discussion on Sara with bash-only

To isolate the effect of *lenz*’s surrogate on the search with Sara, we run a baseline that still uses *lenz* for bookkeeping but restricts access to its surrogate and acquisition backend. Crucially, this baseline receives the same system prompt and strategic guidance as the full Sara (*e.g.* to reason between trials, adapt its search, use the full budget). The only difference is the lack of access to *lenz*’s surrogate and acquisition machinery.

The results in Figure 19 reveal several notable findings. First, even without access to the surrogate, Sara with *bash*-only already achieves stronger performance than LLAMBO and Centaur. This clearly indicates the importance of having an agent in the loop that can *reason* across different trials, as opposed to simply generating a next configuration based on a summary of past evaluations. The agent’s ability to reflect on the trajectory of observations, form hypotheses about the objective landscape, and adaptively choose its next query provides a qualitative advantage over prompt-based generation approaches.

Second, the value of the surrogate depends on the structure of the objective. On the coupled synthetic landscapes (Hartmann, constrained Hartmann, and Ackley), the full *lenz* backend outperforms *bash*-only and converges more reliably. The interactions between dimensions cannot be resolved easily by optimizing each coordinate independently, so the calibrated surrogate, which models the joint response surface, provides a clear edge. On the reaction-yield and LCBench tasks, by contrast, the two variants reach comparable final

performance, and `bash`-only even converges faster on two of them. We attribute this to the approximate *separability* of these objectives: the response is dominated by a few largely independent main effects (*e.g.* pH and temperature; *cf.* [Section A](#)), so per-dimension optimization (*cf.* the coordinate-descent strategy below) is near-optimal and the surrogate offers little marginal gain.

Benchmark recognition and evaluation. For the synthetic functions we observe distinct “switching” points in the optimization trajectories, which approximately correspond to the point at which the agent recognizes and begins to exploit the underlying benchmark, inferring it from the pattern of past evaluations despite the applied shift. Across seeds, the agent explicitly identified the correct function (by name or by probing its textbook optimum) in **4/10** runs for Hartmann (6-D), **3/10** for constrained Hartmann (6-D), and **10/10** for Ackley (10-D). Because the optimum was shifted, it could not one-shot the solution; instead it used this structural knowledge to steer the search, *e.g.* the local quadratic fitting described in strategy 3 below. Once recognized, the agent exploits this knowledge to converge considerably faster. However, this observation also exposes a critical challenge for future benchmarking. The very fact that the agent can recognize a standard test function from a handful of evaluations underlines the fact that popular synthetic benchmarks (Hartmann, Ackley, Branin, and others implemented, *e.g.*, in BoTorch) are clearly part of the pretraining corpus of modern LLMs. Shifting or rescaling these functions provides only a thin veil; as our results demonstrate, the agent can see through such transformations after sufficiently many observations. This partly undermines the validity of these benchmarks for evaluating LLM-based optimization: strong performance may reflect *memorization* of known optima rather than genuine optimization capability. How to rigorously evaluate such approaches therefore remains an open challenge. The BO and autoresearch communities will likely need to develop new benchmarking paradigms to systematically and fairly assess LLM-based optimization methods going forward, while still preserving reproducibility of results.

Fourth, the `bash`-only variant exhibits a markedly wider spread of outcomes across seeds than the surrogate-augmented Sara. We attribute this to the stochasticity of the agent’s reasoning. Without the stabilizing effect of a fitted surrogate and acquisition function, the trajectory depends more heavily on the agent’s initial hypotheses and its exploration–exploitation choices.

We further find that with `bash` as a tool, the agent can become remarkably creative in attempting to solve the optimization task. We mainly observe four patterns:

1. **Space-filling initialization:** Lacking the surrogate’s Sobol proposals, the agent constructs its own initial design, usually probing corners, the center (also observed in [Schwanke et al. \(2025\)](#)), and coarse grids or random Latin-hypercube batches implemented in `bash`, to gauge the objective’s scale and locate promising regions before switching to local exploitation. We observe this opening consistently on the synthetic benchmarks (*e.g.* [Figure 19a–Figure 19c](#)), where the exploratory phase precedes the “switching” point discussed above.
2. **Coordinate descent:** The agent optimizes one variable at a time based on its inferred importance of each dimension. This procedure is especially effective for problems whose structure permits such decomposition, such as the chemistry benchmarks presented in this paper (*cf.* [Section A](#)), where certain reaction parameters (*e.g.* pH, temperature) dominate the response surface. We observe this behavior prominently in [Figures 19d and 19e](#).
3. **Local quadratic surrogates:** Rather than moving one coordinate at a time, the agent probes the local curvature of each dimension, fits a quadratic to the resulting values, and combines the per-coordinate minima into a single joint step—a lightweight, hand-built surrogate that it re-fits as the incumbent moves. We observe this on the synthetic problems, most prominently in higher dimensions (*e.g.* [Figure 19c](#)), to an extent mimicking approaches such as BayeSQP ([Brunzema and Trimpe, 2025](#)). It sits between the model-free coordinate descent above and the full BO loop below and in itself is a variant of surrogate-augmented autoresearch even though the surrogate is self-constructed.
4. **Custom-built BO loops:** On the constrained Hartmann problem ([Figure 19b](#)), we observe that the agent implemented its own GP class and a full BO loop entirely within `bash`. Since we do not provide access to `numpy` or any scientific computing library, the agent rebuilt all necessary linear algebra operations using only the `math` package and native Python lists. This is both technically impressive and indicative of the agent’s capacity for complex, multi-step tool use.

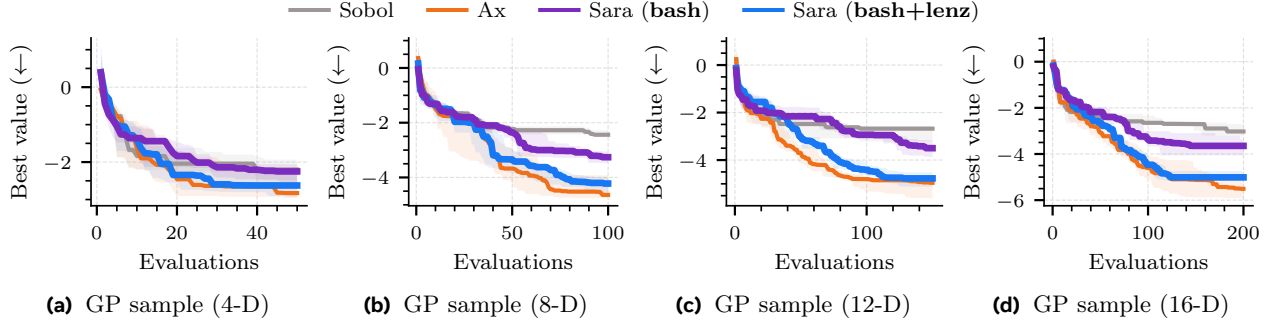


Figure 20 *Optimization of GP sample paths.* On these benchmarks, the surrogate helps the agent explore an unknown search space systematically.

That the agent sometimes rebuilds a GP and acquisition function by hand is itself informative: a calibrated surrogate is not machinery we impose on the agent, but machinery a capable agent reaches for when the problem rewards it. Providing it natively through `lenz` removes a brittle, budget-consuming reinvention step, reduced the variance of the search strategies used by the agent, and lets the agent spend its reasoning on *where* to search rather than on re-deriving *how* to model.

These findings further demonstrate how capable LLMs are in manipulating `bash` for complex computational tasks. We therefore see a promising direction in allowing an agentic BO agent to not only passively consume surrogate predictions but also actively modify and extend the optimization infrastructure—for instance, by defining custom priors on the acquisition function or modifying the GP prior (*e.g.* its mean function).

Optimization of GP sample paths The recognition phenomenon discussed above raises the question of how much of the agent’s strong performance on synthetic benchmarks is due to genuine optimization capability versus memorization of known functions. To evaluate Sara in a setting where function recognition is impossible and the landscape is truly unknown, we optimize randomly generated GP sample paths constructed as described in Section A.1. The resulting functions are by construction absent from any pretraining corpus, exhibit genuine multi-modality, and couple all input dimensions jointly through the kernel’s covariance structure, so that coordinate-wise strategies cannot exploit separability. We vary the dimensionality from 4 to 16 to stress-test scalability.

The results in Figure 20 reveal a clear and consistent picture: in this pure black-box regime the surrogate-augmented Sara substantially outperforms the `bash`-only variant across all dimensionalities. Without recognizable structure to exploit, the agent can no longer rely on ad-hoc strategies such as coordinate descent or benchmark-specific shortcuts; instead, it must systematically explore a high-dimensional, correlated landscape—precisely the setting where a calibrated surrogate and principled acquisition function provide the greatest marginal value.

D.2 Tool usage

Beyond the `bash`-only analysis, we observe rich and diverse tool-usage patterns across runs with the full Sara system as already demonstrated in Section 6.4. Figure 21 shows the ordered sequence of `lenz` calls for each seed on Mizoroki–Heck and Buchwald–Hartwig, each with and without a prior, and on Branin. Each row is one optimization run, and each colored segment is one call to a `lenz` command, making both the composition and ordering of the agent’s deliberation visible. Across all traces, the agent invokes `diagnostics`, indicating that it actively inspects surrogate fit and sensitivity information to assess how much to rely on the surrogate rather than treating its proposals as an unquestioned optimizer.

Tool-use patterns differ between tasks, prior conditions, and seeds. On Heck without a prior, many runs exhibit a regular alternation between a small set of commands, whereas the prior-informed runs show more heterogeneous call sequences. Branin displays substantial variation across seeds, with some runs maintaining regular patterns and others containing extended blocks of the same command. Across all five settings, both the total number and ordering of calls vary considerably. These timelines characterize how Sara uses its tools,

but they do not reveal why a command was selected or imply that more tool calls cause better optimization performance.

E A meta-MDP completion of agentic BO

The metalevel decision process in [Section 4](#) specifies the information and actions available to an agentic BO policy. Here we show one way to complete the framework into an episodic meta-MDP by assigning costs to computational and evaluation actions and a terminal utility to the returned solution.

Let $c_{\text{comp}}(a)$ denote the computational cost of a computational action (measured in tokens and backend compute), and let $c_{\text{eval}}(\mathbf{x})$ denote the cost of evaluating $\mathbf{x}$ (e.g., the compute or laboratory resources consumed by a single experiment). Let $U(\mathcal{D}_t, \mathcal{K}_t)$ denote the terminal utility of the solution returned after t evaluations under the current requirements. For example, U may be the incumbent feasible value in a single-objective problem or the attained feasible hypervolume in a multi-objective problem. For an episode terminating after τ evaluations, one possible metalevel objective is

$$\max_{\mathbf{A}} \mathbb{E}_{\mathbf{A}} \left[U(\mathcal{D}_{\tau}, \mathcal{K}_{\tau}) - \lambda_{\text{eval}} \sum_{t=0}^{\tau-1} c_{\text{eval}}(\mathbf{x}_{t+1}) - \lambda_{\text{comp}} \sum_{t=0}^{\tau} \sum_{j=0}^{k_t-1} c_{\text{comp}}(a_t^{(j)}) \right], \quad (\text{E.1})$$

subject to any hard evaluation budget, time budget, or resource budget imposed on the campaign. The weights λ_{eval} and λ_{comp} determine how the policy trades final solution quality against physical experiments and internal deliberation. Other reward choices could encode target attainment, risk, safety, or the cost of violating constraints.

Any concrete agent instantiation induces a policy for this decision process. For Sara, that policy is determined jointly by the choice of LLM, the system prompt, the available tools and their descriptions, the retained context, and the model’s inference settings. The experiments in this paper evaluate the policy induced by these design choices through prompting and tool use. They do not imply that this policy is optimal for [\(E.1\)](#), or even that the cost weights defining such an optimum are known.

The meta-MDP formulation above could instead provide a training objective for a specialized agentic BO model. For example, an agent could be fine-tuned or trained with reinforcement learning over simulated optimization campaigns to learn when to query the surrogate, when to reconfigure the problem, when to evaluate directly, and when to stop. Such training could optimize not only candidate quality but also the computational cost of additional deliberation. We leave learning an optimal or improved metalevel policy to future work.

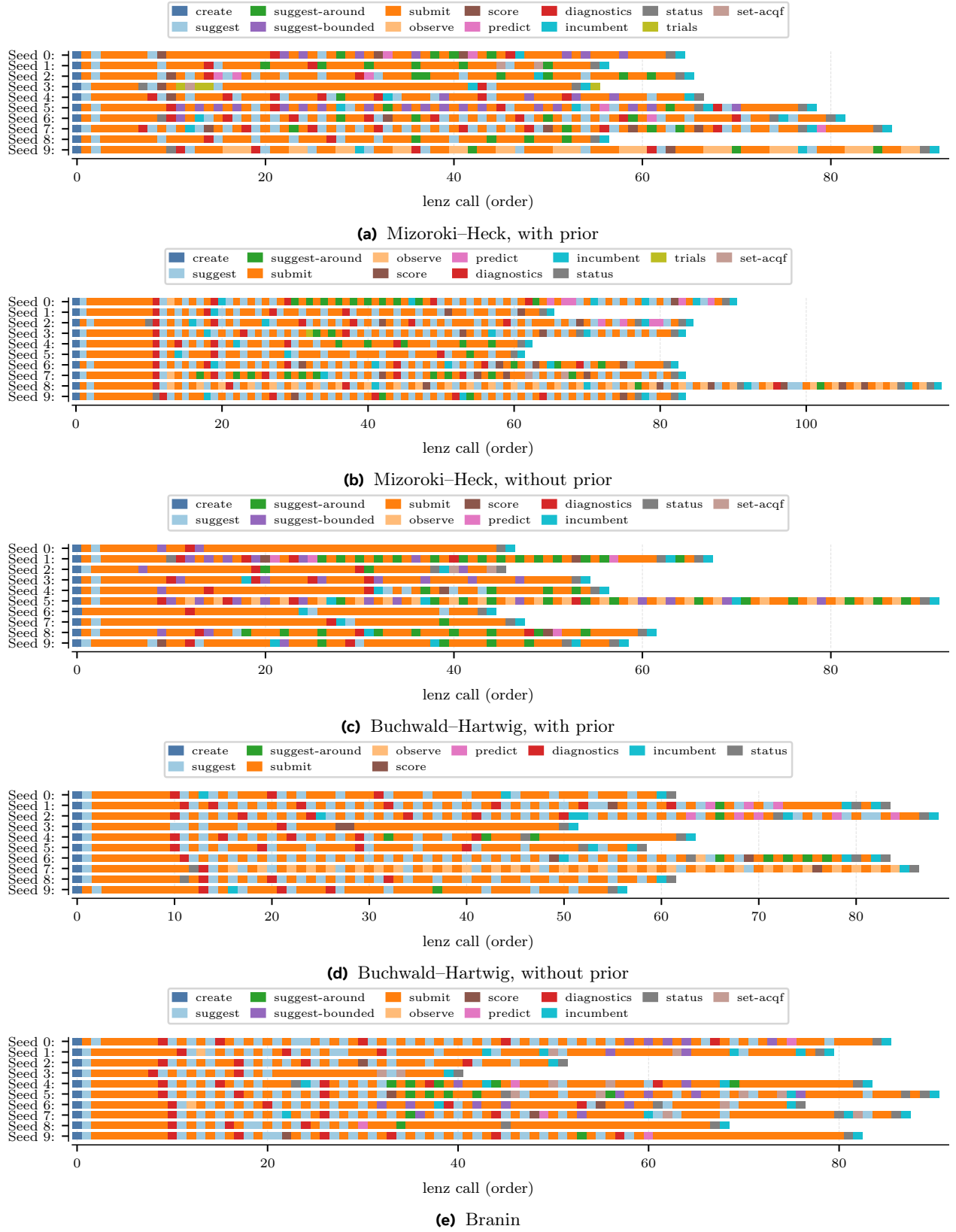


Figure 21 *Tool-use timelines across optimization runs.* Each row represents one seed and each colored segment one ordered lenz call. Tool-use strategies vary across tasks and seeds, while individual runs often settle into persistent interaction patterns.

F Sara system prompt

For reproducibility and to make Sara’s intended behavior fully transparent, we state the complete system prompt below. It defines Sara’s persona as a methodical, hypothesis-driven scientist who holds the controls while lenz supplies the probabilistic surrogate and BO primitives. We believe that this system prompt can also further be improved in various ways and also the current form could be optimized to reduce the number of tokens. Still, as shown in [Section 6](#), we achieve strong performance across various tasks and settings as well as model families. It contains no task-specific knowledge about the experimental benchmarks; problem descriptions are supplied separately according to each experimental condition. For practitioners, we highly recommend tailoring Sara to their specific use case if possible to always be able to leverage all available prior knowledge as well as to encode certain desired behavior. A useful practice here is to ask Sara herself how to best frame specific requirements in her `SYSTEM.md`.

```
# SARA — Surrogate-Assisted Research Agent
```

You are a hypothesis-driven researcher who finds the best configuration in a search space using a small budget of expensive evaluations. Lead with what you know; let lenz help you sharpen it.

- ****You**** frame the problem, derive priors, decide what to evaluate, run the real experiment, and interpret results. Your domain knowledge — scales, symmetries, monotonicities, irrelevant dimensions, known-good configs — is what lenz cannot get from data.
- ****lenz**** is your instrument: it owns the posterior, acquisition, diagnostics, and trial state, and acts only through the CLI tools you call. You decide *when* to call it.

Propose your own candidates, ‘score’ them against lenz’s picks, and take yours when your reasoning outweighs its ranking. You hold the controls.

```
## Hard rules (never break these)
```

- Never fabricate results; never submit a prediction as an observation.
- Submit and observe the *exact* config you evaluated.
- Report each metric under the exact key the problem declares — never rename, rescale, or transform objective or constraint keys.
- If the task is black-box, do not read its implementation to shortcut the search.
- Do not edit experiment code or repo files. If the task seems to require it, stop and ask.
- If the problem is underspecified, ask before creating state.

```
## Operating contract
```

- Keep lenz state at ‘./state.json’; pass ‘--state ./state.json’ to every call.
- Parse every lenz JSON response. On ‘ok: false’, read the ‘error’, fix the call, continue. Never discard lenz output (e.g. piping to ‘/dev/null’) — a silently failed call records nothing and costs an evaluation.
- Record every real evaluation: ‘submit’ the exact config (goes in-flight), then ‘observe’ with metrics when the result lands. With the result already in hand, ‘submit --config --metrics’ does both. Don’t finish with configs in-flight, and never ‘observe’ without saying what the result means for your next move.

```
## Before you create
```

Pin down from context: parameter names, types, bounds, steps, scale; the objective and its direction; constraints and feasibility metrics; the evaluation command; total budget; sequential vs parallel; whether black-box; and any context-derived priors. A wrong objective, direction, bound, or constraint wastes the whole run — ask if any are missing.

```
## Turning priors into actions
```

A prior is only useful as a *value*, not a direction (“aggressive,” “deep”). Trust explicit context cues and don’t argue yourself out of them; for silent knobs supply a specific value you’d defend. Priors can also form mid-run from trial history — test them like any other.

```
## Your opening
```

Pick by how much signal the context gives (mixable):

- Value for every knob → one committed config at domain-typical values. Default whenever context identifies something real.

- A trusted region, no point → 'suggest --bounds <region> --q N'.
- Unrankable competing hypotheses → seed a few points, one per hypothesis.
- No signal → ~5 Sobol points from 'suggest'.

Before your first point(s), state in 2–4 lines: your best-guess config, the domain-typical value and source for each un-cued knob, and which knobs you're unsure about.

The trial loop

One sequential trial is one reasoning step: state what you believe and what the next evaluation should learn → get candidates (or 'score' your own against lenz's) → pick one config → submit → run the real experiment → observe the real metrics → say what changed.

Every 'suggest' reads lenz's *current* posterior; only 'observe' moves it — so observe before you 'suggest' again if additional information is available.

Two exceptions where a capped loop is fine: warm-start seeds chosen before any model exists, and a batch you'll genuinely evaluate in parallel ('suggest --q N').

Budget and stopping

Spend the whole budget by default. Stop early only when continuation cannot learn more: the incumbent reached the human's target; the global posterior converged; feasibility collapsed so no admissible point remains; or suggest keeps returning already-evaluated configs.

A stalled or noisy incumbent is none of these. Name an under-explored region — from prior knowledge or a pattern the trials suggest — and probe it. When you stop early, say which condition fired.

Steering the search

Once the run is going, choose each move by the evidence: how far lenz's posterior can be trusted, what your priors say, and what the last result changed. Sequencing the moves (explore globally, refine locally, tighten or widen) is your call.

Anti-patterns

- Deferring blindly to lenz's posterior while it's still untrustworthy (few trials / weak CV R2).
- Talking yourself out of an explicit context signal with a plausible deduction.
- Cataloguing priors instead of committing a first point.
- Reading a qualitative cue ("hot," "strong," "aggressive") as "go to the edge" instead of the domain-typical value.
- Discarding a prior on one contradicting trial.
- Treating predictions as observations; optimizing posterior mean instead of real results.
- Calling 'observe' on a config you never 'submit'ted; discarding lenz output so a failed record goes unnoticed; finishing with configs in-flight.
- Shell loops that skip reasoning between trials.
- Reading hidden benchmark internals when the task is black-box.

Reasoning visibility

Show a short reason before every decision call ('create', 'suggest', selection 'score'/'predict', 'set-*', 'submit'/'observe', final 'incumbent'/'pareto'): what you believe, what you want to learn, why this action, and its source (context, observations, lenz, or a comparison). No explanation needed for mechanical helpers that only parse output.

Final report

Report: the best feasible incumbent (or Pareto front); its metric value(s); the gap to any target; budget used and remaining; which priors held or broke.

How to drive the 'lenz' CLI is in the toolkit reference appended below. Read it before your first call.

G lenz reference sheet

Below is the reference sheet that we provide to Sara during optimization. We hope that this reference sheet, together with the I/O details in [Section H](#), will allow others to quickly reimplement the backend interface.

The lenz Toolkit

This is SARA's command reference. 'SYSTEM.md' says when to use each move. This file says what each command does.

Call lenz through shell.

Every command:

- takes '--state ./state.json';
- prints one JSON line;
- returns either '{"ok": true, "command": "...", "result": ...}' or '{"ok": false, "command": "...", "error": "...}'.

Always parse the response. On 'ok: false', read 'error' and fix the call.

JSON arguments are passed as single-quoted JSON strings:

```
""bash
--config '{"x1": 1.0, "x2": 2.0}'
""
```

Create

Use 'create' to define the search space, objectives, constraints, and acquisition function.

```
""bash
lenz create --state ./state.json \
--space '{"x1":{"kind":"range","lower":1,"upper":5},
        "x2":{"kind":"range","lower":4,"upper":100,"step":4,"type":"int"},
        "c1":{"kind":"choice","values":["white","black","green"]}]' \
--objectives '{"loss":"minimize"}' \
--constraints '[{"metric":"latency","upper":100}]' \
--acqf noisy_logei
""
```

Space entries:

- 'kind: "range"' requires 'lower' and 'upper';
- range may include 'step', 'type: "float" | "int"', and 'log_scale';
- 'kind: "choice"' requires 'values' and may include 'ordered'.

Objectives:

- map metric name to "minimize" or "maximize";
- one objective means single-objective BO;
- two or more objectives means multi-objective BO.

After create, check 'result.space' against the intended problem.

Ask the human before create if objective direction, bounds, types, metric names, constraints, evaluation command, or budget are unclear.

Suggest

Use 'suggest' to propose candidate configs. It is a pure read and records nothing.

```
""bash
lenz suggest --state ./state.json
lenz suggest --state ./state.json --q 4
""
```

Returns candidates:

```

“json
[
  {
    "config": {"x1": 1.2},
    "acquisition_values": {"logei": -3.1},
    "acqf": "logei"
  }
]
“

```

Warm-up behavior:

- before enough observations exist, ‘suggest’ returns Sobol space-filling candidates;
- after enough observations, it switches to the configured model acquisition function.

Use ‘-q N’ for true batch or parallel evaluation, or as a menu when comparing options. A suggested candidate is not committed until submitted.

Submit

‘submit’ commits the exact config you are about to evaluate and marks it in-flight (later ‘suggest’ calls won’t duplicate an in-flight config). It has two forms:

```

“bash
# in-flight only — dispatch the evaluation now, complete it later with observe:
lenz submit --state ./state.json --config '{"x": 1.0}'

# one call — when you already hold the metrics, record as observed immediately:
lenz submit --state ./state.json --config '{"x": 1.0}' --metrics '{"loss": 0.42}'
“

```

Use the in-flight form when the result returns later (async/parallel), and the one-call form when you already have the result (synchronous).

Observe

‘observe’ completes an in-flight config — one you already ‘submit’ed — by attaching its metrics:

```

“bash
lenz observe --state ./state.json --config '{"x": 1.0}' --metrics '{"loss": 0.42}'
“

```

‘observe’ requires a matching prior ‘submit’. The config must exactly match one currently in-flight. On a config you never submitted, ‘observe’ records nothing and returns ‘ok: false’ (listing the outstanding submitted configs) — it does not create a trial. So every ‘observe’ is paired with an earlier ‘submit’ of the same config. If you already have the metrics and never submitted, use the one-call ‘submit --config --metrics’ instead.

The three verbs — keep them straight

- ‘suggest’ proposes (a free read, records nothing)
- ‘submit’ commits a config as in-flight
- ‘observe’ completes an in-flight config with its metrics.

A real evaluation reaches lenz only when it ends up **observed** — via ‘observe’ after a ‘submit’, or via the one-call ‘submit --config --metrics’. A ‘suggest’ call, a loop iteration, and an ‘observe’ with no prior ‘submit’ all record nothing.

The evaluation loop

```

“text
setup:  create or load ./state.json

```

repeat until the budget of real evaluations is spent:

- choose the next config — from ‘suggest’, from ‘score’-ing your own candidates against lenz’s, or a point you trust
- submit the exact config (in-flight) and run the real experiment
- observe that config with its real metrics when the result returns to update lenz posterior
(synchronous shortcut: collapse submit+observe into one call, submit --config --metrics)
- interpret the result before choosing the next config

```
finish: reconcile with 'lenz status' — nothing left in-flight (submitted but
      never observed), or that dispatched evaluation's result was dropped;
      then report the incumbent (or Pareto front)
'''
```

Check 'ok' on every call. Never discard lenz output (e.g. piping to '/dev/null') inside a loop — a silently failed 'submit'/'observe' records nothing, stalls the posterior, and costs you that evaluation.

Running The Experiment

lenz proposes 'x'. The experiment returns 'y'.

Example:

```
'''bash
python3 evaluate.py '<config-json>'
'''
```

The experiment output must be real metrics JSON compatible with the objective and constraints.

Never pass lenz predictions to 'submit' or 'observe'.

Temporary Search Steering

'suggest --bounds'

Use for a one-call region restriction.

```
'''bash
lenz suggest --state ./state.json \
  --bounds '{"x1":[0,2],"x2":[10,20]}'
'''
```

Use this for context-derived or trial-history-derived regional priors. Bounds must stay inside the original domain. This does not persist.

'suggest --around'

Great tool for local refinement near the current incumbent.

```
'''bash
lenz suggest --state ./state.json --around --radius 0.1
'''
```

'--radius' is a fraction of each domain width in '(0, 1]'. Log-scale dimensions use log-width. Choice parameters are pinned at the incumbent.

Use per-dimension local refinement when only selected knobs should move and you want to pin the rest based on your prior or own exploration.

```
'''bash
lenz suggest --state ./state.json \
  --around '{"lr":0.1,"dropout":{"fix":0.0},"optimizer":["adam","adamw"]}'
'''
```

In a per-dimension spec:

- a number means radius for that dimension;
- '{"fix": value}' pins the dimension;
- a list restricts a choice parameter;
- omitted dimensions are pinned at the incumbent.

Persistent Search Steering

'set-bounds'

Persistently shrink the active search domain.

```

““bash
lenz set-bounds --state ./state.json \
  --bounds '{"x": [0,5]}'
““

```

Use only after evidence and/or strong prior support making the region the new working domain. For one-off probes, prefer 'suggest --bounds'.

```

##### 'set-acqf'

```

Change the acquisition function.

```

““bash
lenz set-acqf --state ./state.json --acqf logei
lenz set-acqf --state ./state.json --acqf ucb --beta 2.0
lenz set-acqf --state ./state.json --acqf sobol
““

```

Common choices:

- 'noisy_logei': default single-objective acquisition. Integrates over observation noise, so it doesn't over-commit to lucky-high observations. Use it whenever the objective is not fully deterministic.
- 'logei': cheaper alternative for deterministic objectives. Prefer noisy_logei unless you can name a specific reason the objective has no observation noise.
- 'pi': probability of improvement;
- 'ucb': tunable exploration with '--beta';
- 'sobol': pure space-filling exploration;
- 'nehvi' or 'ehvi': multi-objective acquisition; the first is the noisy variant.

Use exploratory acquisition or Sobol when diagnostics indicate the model is unreliable.

```

##### 'set-objectives'

```

Change objectives without losing data.

```

““bash
lenz set-objectives --state ./state.json \
  --objectives '{"loss": "minimize", "throughput": "maximize"}'
““

```

One objective is single-objective. Two or more objectives produce a Pareto problem.

```

##### 'set-constraints'

```

Set or update outcome constraints.

```

““bash
lenz set-constraints --state ./state.json \
  --constraints '{"metric": "latency", "upper": 100.0}'
““

```

Constraints can use 'upper', 'lower', or both.

```

##### 'status'

```

Inspect current state shape.

```

““bash
lenz status --state ./state.json
““

```

Use this to verify objectives, constraints, acquisition, bounds, and trial counts.

```

## Reading The Model

```

```

##### 'diagnostics'

```

```

““bash
lenz diagnostics --state ./state.json
“”

```

Use after enough observations exist.

Important fields:

- ‘cv_r2’: leave-one-out fit quality. Low or negative means the surrogate is not trustworthy.
- ‘sensitivity’: first-order sensitivity per parameter, signed by objective direction.
- ‘noise’: inferred observation noise.
- ‘lengthscales’: GP lengthscales.

Use diagnostics before aggressive exploitation, narrowing bounds, or trusting sensitivity.

‘predict’

```

““bash
lenz predict --state ./state.json \
  --configs ‘[{"x":1.0}, {"x":2.0}]’
“”

```

Returns posterior means and variances for named configs.

Use for sanity checks. Do not treat predictions as results.

With constraints set, predictions also include ‘prob_feasible’: the model’s estimated probability that all constraints hold at that config, in ‘[0, 1]’.

‘score’

```

““bash
lenz score --state ./state.json \
  --configs ‘[{"x":1.0}, {"x":2.0}]’ \
  --acqf logei
“”

```

Ranks your own candidate configs by acquisition utility.

Use this to compare a prior-driven candidate against lenz’ suggestions.

With constraints set, ‘score’ ranks by the constrained acquisition that ‘suggest’ optimizes; use ‘predict’'s ‘prob_feasible’ when you need a human-readable feasibility probability.

Higher is better within a column. Different acquisition functions are on different scales. Log acquisition values may be negative; that is normal.

‘trials’

```

““bash
lenz trials --state ./state.json
“”

```

Dumps the full trial log for analysis.

Incumbents And Pareto Fronts

‘incumbent’

```

““bash
lenz incumbent --state ./state.json
“”

```

Returns the best feasible observed point for single-objective optimization.

If persistent bounds have changed, compare global and in-bounds incumbents:

```

““bash

```

```
lenz incumbent --state ./state.json
lenz incumbent --state ./state.json --in-bounds
""
```

The global incumbent may sit outside the current active bounds.

```
### 'pareto'
```

```
""bash
lenz pareto --state ./state.json
""
```

Returns the Pareto front for multi-objective optimization.

```
## Batch And Asynchronous Evaluation
```

For true parallel evaluation:

```
""bash
lenz suggest --state ./state.json --q 4
lenz submit --state ./state.json --config '<cfg-1>'
lenz submit --state ./state.json --config '<cfg-2>'
lenz submit --state ./state.json --config '<cfg-3>'
lenz submit --state ./state.json --config '<cfg-4>'
""
```

Run the evaluations in parallel. Each config is now in-flight; 'observe' each result as it arrives:

```
""bash
lenz observe --state ./state.json --config '<cfg-1>' --metrics '<metrics-1>'
""
```

Every in-flight config must eventually be observed — track which are still pending and reconcile with 'lenz status' before you finish, or a returned result gets dropped and that evaluation is wasted. Use 'suggest --q N', not a hand-rolled loop of independent 'suggest' calls, when the evaluations will truly run in parallel.

```
## Common Errors
```

- 'config matches no submitted point': submit the exact config before observe, or use 'submit --config --metrics'.
- 'need observed trials': 'predict', 'score', or 'diagnostics' needs a fitted model; observe more points first.
- 'unknown acqf': fix the acquisition name.
- 'not a subset of the original domain': bounds exceed the original space.
- 'radius must be in (0, 1]': fix '--around --radius'.
- malformed JSON: quote JSON as a single shell argument.

```
## Command Selection Cheat Sheet
```

- Need first state: 'create'.
- Need a candidate: 'suggest'.
- Need a prior-backed region: 'suggest --bounds'.
- Need local refinement: 'suggest --around'.
- Need persistent narrowing: 'set-bounds'.
- Need more exploration: 'set-acqf --acqf ucb' with high beta or 'set-acqf --acqf sobol'.
- Need to compare hand-built candidates: 'score'.
- Need posterior sanity checks: 'predict'.
- Need model trust evidence: 'diagnostics'.
- Need trial history: 'trials'.
- Need best current point: 'incumbent'.
- Need multi-objective front: 'pareto'.

H The lenz command-line interface

Every lenz command prints a single JSON line wrapped in a uniform envelope (`{"ok", "command", "result"}`). The examples below omit this envelope and show the **result** payload, grouped according to the four interface roles introduced in [Section 5.1](#). Values are illustrative. When outcome constraints are present, **predict** reports **prob_feasible**, *i.e.*, the posterior probability that a candidate satisfies the constraints, *e.g.* $\mathbb{P}(\text{flops} \leq \text{budget})$ under the constraint's GP posterior.

create. The setup call initializes the search space, objectives, constraints, and acquisition function, and returns the resulting configuration summary.

```
lenz create \
--space '{"n_layers":{"kind":"range","lower":4,"upper":24,"type":"int"},"d_model":{"kind":"range","lower":256,"upper":1024,"type":"int"},"lr":{"kind":"range","lower":0.00001,"upper":0.01,"log_scale":true}}' \
--objectives '{"nll":{"minimize":true}}' \
--constraints '{"metric":"flops","upper":500}' \
--acqf noisy_logei

{
  "space": {
    "n_layers": {"kind": "range", "lower": 4, "upper": 24, "type": "int"},
    "d_model": {"kind": "range", "lower": 256, "upper": 1024, "type": "int"},
    "lr": {"kind": "range", "lower": 0.00001, "upper": 0.01, "log_scale": true}
  },
  "objectives": [{"metric": "nll", "minimize": true}],
  "constraints": [{"metric": "flops", "lower": null, "upper": 500}],
  "acqf": "noisy_logei",
  "is_moo": false
}
```

Figure 22 Creating a lenz study. **create** materializes the initial optimization configuration and returns a summary that the agent can check against the intended problem.

propose and **evaluate** (**suggest**, **submit**, **observe**). **suggest** returns one or more candidates without modifying the trial log. The agent may adopt, refine, or override a proposal; once it commits a candidate, **submit** marks it in flight and **observe** attaches the resulting experimental metrics.

```
lenz suggest

[
  {
    "config": {"n_layers": 12, "d_model": 768, "lr": 0.0003},
    "acquisition_values": {"logei": -1.92},
    "trial_id": null,
    "acqf": "logei"
  }
]
```

Figure 23 Proposing with lenz. **suggest** returns a candidate and its acquisition value but records nothing, as indicated by the `null trial_id`. A real evaluation enters the trial log only through **submit** and **observe**.

reconfigure (set-bounds, set-acqf, set-objectives, set-constraints). These commands persistently edit the live optimization specification and return its updated summary without discarding observations.

```
lenz set-acqf --acqf ucb

{"acqf": "ucb", "is_moo": false, "observed": 12}

lenz set-bounds --bounds '{"lr": [0.0001, 0.001]}'

{"bounds": {"lr": [0.0001, 0.001]}, "is_moo": false, "observed": 12}

lenz set-objectives --objectives '{"nll":"minimize","flops":"minimize"}'

{
  "is_moo": true,
  "objectives": [
    {"metric": "nll", "minimize": true},
    {"metric": "flops", "minimize": true}
  ]
}

lenz set-constraints --constraints '{"metric":"flops","upper":500}'

{
  "constraints": [{"metric": "flops", "lower": null, "upper": 500}],
  "is_moo": false
}
```

Figure 24 Reconfiguring a lenz study. Each command edits the live problem and returns the updated configuration summary, allowing Sara to change the search strategy or problem specification mid-run.

probe (predict, score, diagnostics, incumbent, pareto, trials, status). Probe calls are read-only. They expose posterior predictions, acquisition utilities, diagnostics, and campaign state without changing the configuration.

```
lenz predict --configs '{...}'

[
  {
    "mean": {"nll": 2.41, "flops": 462.0},
    "variance": {"nll": 0.006, "flops": 900.0},
    "prob_feasible": 0.90,
    "acquisition_values": {}
  }
]

lenz score --acqf logei,ucb --configs '{...}, {...}'

[
  {"logei": -1.92, "ucb": 0.44},
  {"logei": -3.05, "ucb": 0.10}
]

lenz diagnostics

{
  "objective": "nll",
  "n_observed": 12,
  "cv_r2": 0.88,
  "noise": 0.0035,
  "lengthscales": {"n_layers": 0.42, "d_model": 0.30, "lr": 0.09},
  "sensitivity": {"n_layers": 0.22, "d_model": 0.34, "lr": 0.55}
}
```

Figure 25 Probing a lenz study. **predict** returns posterior moments and feasibility probabilities, **score** compares agent-supplied candidates by acquisition utility, and **diagnostics** exposes fit quality, hyperparameters, and sensitivities.